ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

ĐỒ ÁN TỐT NGHIỆP

PHÁT TRIỂN PLATFORM EDGE-AI DỰA TRÊN FPGA SỬ DỤNG RISC-V

NGÀNH: KỸ THUẬT MÁY TÍNH
HỘI ĐỒNG: XX

Người hướng dẫn:

PGS. TS. Phạm Quốc Cường	- Trường ĐHBK
<Name>	- <affiliation>

Người Phản biên:

Assoc. Prof. Dr. <name>	- HCMUT

Sinh viên thực hiện:

Nguyễn Trung Tân	- 2213063
Trần Minh Tâm	- 2213039

THÀNH PHỐ HỒ CHÍ MINH - THÁNG 5 NĂM 2026

This thesis is dedicated for our parents and our instructors at HCMUT.

MỤC LỤC

Danh sách hình vẽ **ix**

Danh sách bảng **xi**

Lời cảm ơn **xiii**

Tóm tắt **xv**

1 Giới thiệu đề tài **1**

1.1 Mục tiêu đề tài 2
1.2 Lý do lựa chọn 4
1.3 Giới hạn và phạm vi đề tài 6
1.4 Cấu trúc báo cáo. 8

2 Cơ sở lý thuyết **9**

2.1 Kiến trúc tập lệnh RISC-V 9
2.1.1 Lịch sử hình thành 9
2.1.2 Các đặc điểm nổi bật. 10
2.1.3 Cấu trúc tập lệnh cơ sở và các phần mở rộng 10
2.1.4 So sánh kiến trúc RISC và CISC 12
2.1.5 Các phần mở rộng tiêu chuẩn (Standard Extensions) 13
2.1.6 Sự phù hợp của RISC đối với Edge AI 14
2.2 Lõi vi xử lý CV32E40P 14
2.2.1 Tổng quan và lý do lựa chọn 14
2.2.2 Cấu trúc đường ống lệnh 15
2.2.3 Giao tiếp Open Bus Interface (OBI) 16
2.3 Nền tảng phần cứng fpga và system-on-chip. 17
2.3.1 Công nghệ fpga (field programmable gate array) . . . 17
2.3.2 System on Chip - SoC 18
2.3.3 nền tảng Kria kv260 vision ai 18
2.3.4 Nguồn dữ liệu đầu vào cho hệ thống 19

2.4 tổng quan chuẩn giao tiếp axi 19
2.4.1 Lịch sử hình thành và sự hội tụ giữa ASIC và FPGA . . 19
2.4.2 Cơ sở lý thuyết và Cơ chế hoạt động của AXI4 20
2.4.3 Phân loại các biến thể AXI trên nền tảng Xilinx 22
2.4.4 Tích hợp hệ thống 23
2.5 tổng quan về trí tuệ nhân tạo (ai) 25
2.5.1 Các Mô hình AI (AI Models) 25
2.5.2 Training và Inference 25
2.5.3 Phân loại AI và Mối liên hệ với Deep Learning. 26
2.5.4 Edge AI 26
2.6 Mạng Nơ-ron Nhân tạo (Artificial Neural Networks) 27
2.6.1 Các kiến trúc Mạng Nơ-ron phổ biến 27
2.6.2 Mạng Nơ-ron Tích chập (CNN) 27
2.6.3 Kỹ thuật Lượng tử hóa (Quantization) 28
2.6.4 Các kỹ thuật tối ưu hóa khác 29

3 Các công trình nghiên cứu liên quan 31
3.1 Pulpissimo 31
3.1.1 Tổng quan 31
3.1.2 Các đặc điểm kỹ thuật nổi bật 31
3.2 Mô hình hiệu quả cho thiết bị biên. 34
3.3 Gia tốc phần cứng trên Tập lệnh mở rộng RISC-V (ISA Extensions) 34
3.4 Lượng tử hóa và huấn luyện mạng nơ-ron 36
3.4.1 Cơ chế Lượng tử hóa Affine (Affine Quantization Scheme) 36
3.4.2 Suy luận chỉ dùng số học nguyên (Integer-Arithmetic-Only Inference) 37

4 Thiết kế kiến trúc hệ thống 39
4.1 Tổng quan kiến trúc Platform 39
4.1.1 Layer 1: Hardware Layer. 40
4.1.2 layer 2: software / runtime layer 40
4.1.3 layer 3: application layer 41
4.2 kiến trúc dị thể. 41
4.3 Thiết kế Processing System 44
4.3.1 Cấu hình chi tiết các phân hệ. 44

4.4 Thiết kế Hệ thống Giao tiếp và Bộ nhớ (Interconnect & Memory Hierarchy) 46
4.4.1 Cơ chế AXI SmartConnect (PS-PL Bridge) 46
4.4.2 Quy hoạch không gian bộ nhớ (System Memory Map) 48
4.5 Thiết kế Programmable Logic 51
4.5.1 Kiến trúc tổng thể Phân hệ PULP (PULP Subsystem). 51
4.5.2 Cấu hình Nhân Vi xử lý (RISC-V Core Configuration) 52
4.5.3 Hệ thống Bộ nhớ TCDM và Interconnect 52
4.5.4 Cơ chế Truy cập Bộ nhớ Trực tiếp (uDMA) 53
4.6 Thiết kế Khối Tăng tốc phần cứng (CNN Accelerator) 54
4.6.1 Kiến trúc tổng quát khối Accelerator 54
4.6.2 Các thành phần chức năng chi tiết 54
4.6.3 Đặc tính thiết kế và tối ưu cho vgg-tiny 55
4.7 Mô hình AI mục tiêu: vgg-tiny. 57
4.7.1 Đặc tả kiến trúc vgg-tiny 57
4.7.2 So sánh vgg-tiny với các mô hình thông dụng 58
4.7.3 Lộ trình mở rộng (Tier Roadmap) 58
4.7.4 Lượng tử hóa (Quantization) 59
4.7.5 Tổ chức Dữ liệu trong Bộ nhớ. 59
4.8 Luồng thực thi và Giao thức bắt tay 59

5 Hiện thực và kiểm thử hệ thống 63
5.1 Tổng quan hướng hiện thực 63
5.2 Huấn luyện và lượng tử hóa mô hình AI 65
5.3 Hiện thực khối gia tốc phần cứng CNN 66
5.3.1 Tổng quan kiến trúc Conv_CNN v2.0 66
5.3.2 Tham số hóa và package `conv_pkg.sv` 66
5.3.3 Tích hợp tổng thể qua AXI 67
5.3.4 Đánh giá khối tăng tốc CONV-CNN 67
5.4 Hiện thực RISC-V Soft-core 68
5.4.1 Tinh chỉnh CV32E40P Core 68
5.4.2 Hiện thực cầu nối giao tiếp OBI sang AXI4 (OBI to AXI) 69
5.5 Phát triển Firmware. 71
5.5.1 Pipeline huấn luyện trên Host PC 71

5.5.2 Trình biên dịch tự sinh `layer_table.h` và `weights.bin` 71
5.5.3 Firmware RISC-V 71
5.5.4 Host Controller 71
5.6 Tích hợp Block Design trong Vivado 72
5.6.1 Bản đồ bộ nhớ và Address Editor 75
5.6.2 Đồng bộ ARM ↔ RISC-V qua D-BRAM dual-port . . . 76
5.7 Quy trình kiểm thử 77
5.7.1 Kiểm thử đơn vị (Unit Testbench) 77
5.7.2 Kiểm thử tích hợp IP 77
5.7.3 Đối chiếu với golden vector từ TFLite Interpreter . . . 77
5.7.4 Kiểm thử trên board KV260 77

6 Kết quả 79
6.1 Kết quả huấn luyện model 79
6.1.1 Cấu hình thí nghiệm 79
6.1.2 Tập Cats vs Dogs 79
6.1.3 Tập INRIA Person 81
6.1.4 Đánh giá ảnh hưởng của lượng tử hóa INT8 81
6.2 Tổng hợp phần cứng 82
6.2.1 Tài nguyên FPGA 82
6.2.2 Báo cáo timing 84
6.2.3 Power estimation 85
6.3 Kết quả chạy trên mạch thực tế 86
6.3.1 So sánh với related work 86

7 Kết luận và Hướng phát triển 89
7.1 Kết luận 89
7.1.1 Đạt được 89
7.1.2 Hạn chế của hệ thống 90
7.2 Hướng phát triển 91

Tài liệu tham khảo 93

DANH SÁCH HÌNH VẼ

2.1 Sơ đồ AXI (AMBA) 20
2.2 Các kênh giao tiếp (AMBA) 21

3.1 Kiến trúc Pulpissimo 32
3.2 Quy trình lượng tử hóa và huấn luyện mạng nơ-ron theo Jacob et al. 36

4.1 Stack kiến trúc hệ thống Edge-AI 39
4.2 Kiến trúc hệ thống tổng quan 42
4.3 Sơ đồ khối hệ thống giao tiếp AXI giữa PS và PL. 46
4.4 Sơ đồ kiến trúc nội bộ của Phân hệ PULP trên FPGA. 51
4.5 Sơ đồ khối chức năng chi tiết của bộ tăng tốc CNN (Accelerator Core Architecture). 54
4.6 Sơ đồ quy trình 61

5.1 Sơ đồ tổng quan 64
5.2 Sơ đồ datapath conv_cnn IP: từ AXIS in qua line buffer, window, MAC array, requantize đến AXIS out. 66
5.3 Báo cáo chạy khối CNN 68
5.4 Kết quả chạy simulate khối CNN 69
5.5 Sơ khối minh họa hệ thống 73
5.6 Thiết kế khối trong vivado 74

6.1 Đường cong loss và accuracy theo epoch. 80
6.2 Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic) 82
6.3 Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic) 83
6.4 Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic) 84
6.5 Báo cáo kiểm tra luật thiết kế (DRC Report) 85
6.6 Báo cáo thời gian (Timing Summary) đảm bảo không có vi phạm Slack 86
6.7 Báo cáo ước tính công suất tiêu thụ (Power Report) của toàn hệ thống 87

DANH SÁCH BẢNG

2.1 Quy ước sử dụng thanh ghi trong RISC-V (ABI) 11
2.2 So sánh đặc điểm kỹ thuật giữa RISC và CISC 13

4.1 Bản đồ bộ nhớ hệ thống (System Memory Map) 48
4.2 Bố cục thanh ghi chia sẻ trong D-BRAM Port B 49
4.3 Thông số cấu hình RISC-V Soft-core 52
4.4 Cấu trúc chi tiết các lớp mạng vgg-tiny (đầu vào 128×128×3, lượng tử hóa INT8) 57
4.5 So sánh vgg-tiny với các mạng CNN phổ biến 58

5.1 Các artifact trao đổi giữa các tầng trong pipeline. 64
5.2 Bản đồ thanh ghi điều khiển S00_AXI (32 bytes, 8 thanh ghi). 67

6.1 Accuracy Tiny-VGG trên tập Cats vs Dogs. 79
6.2 Tài nguyên KV260 sử dụng (sau implementation). 83

LỜI CẢM ƠN

Việc hoàn thành đồ án này không chỉ là kết quả của sự nỗ lực từ phía nhóm thực hiện, mà còn là sự kết tinh từ sự chỉ dẫn tận tình và động viên to lớn của quý thầy cô, gia đình và bạn bè.

Lời tri ân sâu sắc nhất, chúng tôi xin được gửi tới Thầy Phạm Quốc Cường. Thầy không chỉ là người hướng dẫn khoa học, định hướng cho chúng tôi những lối đi đúng đắn trong mê cung kiến thức, mà còn là người truyền lửa, giúp chúng tôi kiên trì vượt qua những rào cản kỹ thuật hóc búa trong suốt quá trình thực hiện đề tài. Những kinh nghiệm quý báu mà Thầy chia sẻ thực sự là hành trang vô giá đối với chúng tôi.

Chúng tôi cũng xin bày tỏ lòng biết ơn chân thành đến Quý Thầy Cô thuộc Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa - ĐHQG TP.HCM. Môi trường học thuật nghiêm túc và nền tảng kiến thức vững chắc mà các Thầy Cô đã dày công vun đắp chính là bệ phóng để chúng tôi có thể tự tin giải quyết các vấn đề trong đồ án này cũng như trong sự nghiệp sau này.

Cuối cùng, xin gửi lời cảm ơn đến gia đình – hậu phương vững chắc nhất, và những người bạn đã luôn sát cánh, chia sẻ khó khăn. Sự ủng hộ của mọi người chính là nguồn động lực tinh thần to lớn giúp chúng tôi đi đến cuối hành trình này.

Xin chân thành cảm ơn!

TÓM TẮT

Trong kỷ nguyên số hóa, Internet vạn vật (IoT) đã chuyển mình từ một khái niệm công nghệ trở thành hạ tầng thiết yếu của đời sống hiện đại. Từ các thiết bị dân dụng như Smart TV đến các mạng lưới cảm biến công nghiệp phức tạp, IoT đang tạo ra một lượng dữ liệu khổng lồ. Xu hướng tất yếu hiện nay là tích hợp Trí tuệ nhân tạo (AI) vào các thiết bị này để tạo nên AIoT (Artificial Intelligence of Things), giúp thiết bị không chỉ "kết nối"mà còn có khả năng "tư duy".

Tuy nhiên, việc triển khai AI trên các thiết bị IoT truyền thống đang vấp phải "nút thắt cổ chai"về kiến trúc. Mô hình phổ biến hiện nay dựa vào việc đẩy dữ liệu lên máy chủ đám mây (Cloud Server) để xử lý tập trung. Mô hình này bộc lộ nhiều điểm yếu chí mạng: rủi ro gián đoạn dịch vụ khi mất kết nối mạng, độ trễ cao không phù hợp cho các ứng dụng thời gian thực, và gánh nặng băng thông lớn.

Về mặt phần cứng, các vi xử lý (CPU) thương mại từ các hãng lớn, dù mạnh mẽ, thường gặp vấn đề về tiêu thụ năng lượng khi phải gồng gánh các tác vụ AI liên tục. Hơn nữa, tính chất đóng kín của các dòng chip này khiến các nhà phát triển gặp khó khăn trong việc tùy biến sâu để tối ưu hóa hiệu năng cho các thuật toán AI cụ thể, đồng thời rào cản chi phí bản quyền cũng là một trở ngại lớn cho đổi mới sáng tạo.

Sự xuất hiện của RISC-V đã mang đến một luồng gió mới cho ngành công nghiệp bán dẫn. Là một kiến trúc tập lệnh mở (Open ISA), RISC-V phá bỏ thế độc quyền của các kiến trúc đóng, trao quyền cho các tổ chức nhỏ, các nhóm nghiên cứu và cá nhân khả năng tự thiết kế các lõi CPU chuẩn hóa. Ưu điểm vượt trội của RISC-V nằm ở tính mô-đun hóa và khả năng mở rộng. Các kỹ sư có thể tự do thêm các tập lệnh tùy chỉnh hoặc sửa đổi vi kiến trúc để phục vụ cho các tác vụ chuyên biệt mà không vướng phải rào cản pháp lý hay chi phí bản quyền đắt đỏ. Đây chính là chìa khóa để giải quyết bài toán hiệu năng/năng lượng cho các thiết bị IoT thế hệ mới.

Dựa trên những phân tích trên, dự án của nhóm tập trung phát triển một nền tảng Edge-AI (AI tại biên) chuyên dụng, lấy kiến trúc RISC-V làm nòng cốt. Mục tiêu của nhóm không chỉ dừng lại ở việc sử dụng một lõi CPU đa năng. Thay vào đó, nhóm tận dụng khả năng tùy biến của RISC-V để thiết kế một hệ thống tích hợp sâu, bao gồm một lõi xử lý trung tâm

được tối ưu hóa kết hợp với một bộ tăng tốc AI tùy chỉnh chạy song song trên FPGA Kria KV260.

Đồ án triển khai đầy đủ bốn lớp đồng thiết kế: *(i)* Lớp toolchain huấn luyện trên Host PC sinh ra layer table và blob trọng số INT8 từ mô hình Keras; *(ii)* Lớp gia tốc dữ liệu (data plane) gồm bộ tăng tốc Conv-CNN v2.0 thiết kế bằng SystemVerilog, hỗ trợ tích chập 3×3 INT8 với khối requantize tuân thủ chuẩn TFLite; *(iii)* Lớp điều phối (orchestrator) là firmware bare-metal chạy trên lõi RISC-V CV32E40P, đọc layer table và lập lịch DMA cho từng layer theo cơ chế ping-pong DDR; *(iv)* Lớp ứng dụng host chạy trên ARM Cortex-A53 với runtime PYNQ + Jupyter notebook, đảm nhận tiền xử lý ảnh, kích hoạt RISC-V và hậu xử lý kết quả. Hệ thống được kiểm chứng qua bài toán phân loại nhị phân với mô hình `vgg-tiny` (5 layer Conv 3×3, ~25.7K tham số INT8) trên hai dataset Cats vs Dogs và INRIA Person. Kết quả thực nghiệm trên KV260 cho thấy hệ thống hoạt động ổn định, xử lý chính xác giao thức bắt tay ARM ↔ RISC-V qua D-BRAM dual-port, và là nền tảng có thể mở rộng cho các kiến trúc mạng phức tạp hơn (VGG-11, ResNet-18) trong các pha phát triển tiếp theo.

1

GIỚI THIỆU ĐỀ TÀI

Sự bùng nổ của AI và rào cản phần cứng Trong thập kỷ qua, thế giới đã chứng kiến sự chuyển mình mạnh mẽ của cuộc Cách mạng Công nghiệp 4.0, trong đó Trí tuệ nhân tạo (AI) đóng vai trò là động lực cốt lõi. Đặc biệt, sự ra đời của các mô hình ngôn ngữ lớn (LLMs) như ChatGPT đã đánh dấu một bước ngoặt lịch sử, đưa AI từ các ứng dụng nghiên cứu hẹp sang kỷ nguyên của AI tạo sinh (Generative AI) với khả năng tác động sâu rộng đến mọi mặt của đời sống xã hội. Tuy nhiên, đi cùng với tiềm năng to lớn đó là những thách thức chưa từng có về mặt hạ tầng tính toán. Các thuật toán AI hiện đại ngày càng trở nên phức tạp, đòi hỏi khả năng xử lý dữ liệu song song khổng lồ và băng thông bộ nhớ lớn. Điều này đã đẩy các kiến trúc vi xử lý truyền thống đến giới hạn của định luật Moore, gây ra các vấn đề nghiêm trọng về tiêu thụ năng lượng và độ trễ khi triển khai trên diện rộng.

Mô hình xử lý AI tập trung trên đám mây hiện nay đang bộc lộ nhiều điểm yếu, bao gồm độ trễ đường truyền, rủi ro về bảo mật dữ liệu riêng tư và sự phụ thuộc vào kết nối Internet liên tục. Do đó, xu hướng công nghệ đang dịch chuyển mạnh mẽ sang tính toán tại biên (Edge Computing). Việc đưa AI xuống chạy trực tiếp trên các thiết bị cuối (Edge devices) đòi hỏi phần cứng không chỉ mạnh mẽ mà còn phải tiết kiệm năng lượng và chi phí thấp – một bài toán khó đối với các dòng chip thương mại đóng kín (proprietary chips) hiện có trên thị trường.

Sự trỗi dậy của RISC-V Giữa bối cảnh "nút thắt cổ chai"về phần cứng đó, sự xuất hiện của kiến trúc tập lệnh mở RISC-V được xem là một cuộc

cách mạng trong ngành bán dẫn. Khác với các kiến trúc đóng như x86 hay ARM vốn đòi hỏi chi phí bản quyền đắt đỏ và không cho phép can thiệp sâu vào thiết kế lõi, RISC-V mang đến sự tự do hoàn toàn. Điểm ưu việt nhất của RISC-V là tính mô-đun hóa và khả năng mở rộng. Nó cho phép các kỹ sư thiết kế có thể thêm vào các tập lệnh tùy chỉnh để tối ưu hóa đặc biệt cho các tác vụ AI cụ thể, giúp tăng hiệu suất xử lý lên nhiều lần trong khi vẫn giữ mức tiêu thụ năng lượng ở mức thấp.

Sự kết hợp RISC-V và công nghệ FPGA Để hiện thực hóa sức mạnh của RISC-V mà không cần tốn kém chi phí sản xuất chip silicon ngay từ đầu, công nghệ FPGA đóng vai trò là nền tảng phát triển lý tưởng. FPGA cho phép chúng ta thiết kế, kiểm thử và tái cấu hình phần cứng một cách linh hoạt như phần mềm. Sự kết hợp giữa khả năng tùy biến của RISC-V và tính linh hoạt của FPGA tạo điều kiện thuận lợi để xây dựng các dòng chip chuyên biệt hóa dành cho AI.

1.1. MỤC TIÊU ĐỀ TÀI

Mục tiêu tổng quát của đồ án là thiết kế, hiện thực và đánh giá một Nền tảng tính toán biên (Edge-AI Platform) hoàn chỉnh trên FPGA Kria KV260, ứng dụng kiến trúc Đa xử lý Bất đối xứng (AMP - Asymmetric Multiprocessing). Hệ thống là sự kết hợp giữa khả năng quản lý linh hoạt của vi xử lý Host (ARM Cortex-A53) và sức mạnh xử lý luồng dữ liệu chuyên biệt của lõi RISC-V, tạo ra một hệ sinh thái toàn diện từ phần cứng đến phần mềm để thực thi các mạng nơ-ron nhân tạo.

Các mục tiêu cụ thể bao gồm:

- **Nghiên cứu kiến trúc SoC và lõi RISC-V:** Phân tích kiến trúc hệ thống xử lý (Processing System) của Zynq UltraScale+ và tổ chức lõi xử lý mã nguồn mở RISC-V CV32E40P. Tùy biến và triển khai thành công lõi RISC-V lên vùng logic khả trình (PL) của FPGA để hoạt động như một bộ gia tốc phần mềm (Software Accelerator) độc lập.

- **Làm chủ và tối ưu giao tiếp Memory Map:** Thiết kế hệ thống bus liên kết (Interconnect) dựa trên chuẩn AMBA AXI4. Trọng tâm là xây dựng cơ chế giao tiếp và đồng bộ hóa dữ liệu tốc độ cao giữa lõi ARM và RISC-V thông qua bộ nhớ chia sẻ (Shared BRAM), giải quyết bài toán nút thắt cổ chai dữ liệu (Data Bottleneck).

- **Phát triển Firmware và Quy trình Đồng thiết kế (HW/SW Co-design):** Phát triển phần mềm quản lý lớp ứng dụng trên hệ điều hành Linux

(Host) và tối ưu hóa firmware C bare-metal cho lõi RISC-V. Ứng dụng các tập lệnh mở rộng để thực thi hiệu quả các phép toán ma trận của mạng CNN nhằm giảm tải cho vi xử lý trung tâm.

- **Pipeline huấn luyện và biên dịch mô hình:** Xây dựng pipeline huấn luyện mạng `vgg-tiny` INT8 trên Host PC, kết hợp với trình biên dịch tự sinh layer table (`layer_table.h`) và blob trọng số (`weights.bin`) – cho phép thay đổi mô hình AI mà không cần tổng hợp lại RTL. Pipeline hỗ trợ hai dataset chuẩn: Cats vs Dogs (Kaggle) và INRIA Person.

- **Tích hợp và Đánh giá hệ thống:** Triển khai toàn bộ nền tảng và kiểm chứng bằng pipeline phân loại nhị phân (binary classification) với mô hình `vgg-tiny` (5 conv layer, INT8) trên ảnh tĩnh đọc từ thẻ nhớ SD. Đánh giá nền tảng dựa trên các tiêu chí: độ chính xác phân loại, độ trễ inference, tài nguyên FPGA tiêu thụ (LUT, FF, BRAM, DSP), công suất tiêu thụ và khả năng mở rộng cho các mô hình lớn hơn ở các pha sau.

1

1.2. LÝ DO LỰA CHỌN

Trong bối cảnh kỷ nguyên số hóa hiện nay, sự bùng nổ của Internet vạn vật (IoT) đã tạo ra một lượng dữ liệu khổng lồ tại các thiết bị đầu cuối. Mô hình xử lý tập trung trên đám mây (Cloud Computing) truyền thống đang dần bộc lộ những hạn chế về độ trễ đường truyền, băng thông mạng và đặc biệt là rủi ro về an toàn thông tin. Do đó, xu hướng chuyển dịch năng lực tính toán từ đám mây xuống biên (Edge AI) đang trở thành một yêu cầu tất yếu. Việc xử lý dữ liệu ngay tại nơi sinh ra dữ liệu không chỉ giúp giảm thiểu độ trễ xuống mức mili-giây, đáp ứng yêu cầu thời gian thực của các hệ thống nhúng tự hành, mà còn đảm bảo quyền riêng tư người dùng khi dữ liệu nhạy cảm không cần phải gửi đi qua môi trường mạng công cộng.

Để đáp ứng nhu cầu tính toán hiệu năng cao tại biên với ngân sách năng lượng hạn hẹp, các kiến trúc vi xử lý đa năng truyền thống đang dần nhường chỗ cho các thiết kế phần cứng chuyên biệt. Trong đó, kiến trúc tập lệnh mở RISC-V nổi lên như một cuộc cách mạng trong ngành công nghiệp bán dẫn. Khác với các kiến trúc đóng và độc quyền (Proprietary ISA) như ARM hay x86, RISC-V cung cấp quyền tự chủ hoàn toàn cho nhà thiết kế, cho phép tùy biến và mở rộng tập lệnh (Custom Extensions) để tối ưu hóa cho các tác vụ đặc thù của AI như tích chập hay tính toán ma trận. Sự linh hoạt này, kết hợp với mô hình mã nguồn mở, đã tạo ra một cộng đồng nghiên cứu và phát triển mạnh mẽ, biến RISC-V thành nền tảng lý tưởng để thử nghiệm các kiến trúc máy tính thế hệ mới.

Để hiện thực hóa và kiểm chứng các thiết kế SoC dựa trên RISC-V cho ứng dụng Edge AI, việc lựa chọn nền tảng phần cứng phù hợp là yếu tố then chốt. Board mạch Kria KV260 Vision AI Starter Kit của AMD-Xilinx được lựa chọn cho đề tài này nhờ sở hữu kiến trúc tính toán không đồng nhất mạnh mẽ của dòng Zynq UltraScale+ MPSoC. Sự kết hợp giữa Hệ thống xử lý (Processing System - PS) dựa trên ARM và Phần logic lập trình được (Programmable Logic - PL) của FPGA tạo điều kiện thuận lợi cho việc đồng thiết kế phần cứng/phần mềm. Nền tảng này cho phép chúng tôi linh hoạt triển khai các lõi RISC-V mềm (Soft-core) song song với việc tận dụng tài nguyên phần cứng dồi dào để xây dựng các bộ tăng tốc AI, từ đó đánh giá chính xác hiệu quả của giải pháp đề xuất trong môi trường thực tế.

Hiện nay, Edge AI trở thành xu hướng tất yếu trong hệ thống nhúng và IoT, do yêu cầu xử lý dữ liệu nhanh, giảm độ trễ và bảo vệ quyền riêng tư. Trong khi đó, RISC-V nổi bật nhờ tính mở, linh hoạt, dễ mở rộng và đang được cộng đồng nghiên cứu rộng rãi để thay thế các kiến trúc độc quyền như ARM.

Board KV260 của AMD-Xilinx tích hợp SoC Zynq UltraScale+ MPSoC, kết hợp CPU ARM và FPGA, cho phép thử nghiệm song song cả phần cứng RISC-V và phần mềm AI, là nền tảng lý tưởng để nghiên cứu Edge AI trên kiến trúc mở.

1

1.3. GIỚI HẠN VÀ PHẠM VI ĐỀ TÀI

Do sự phức tạp của việc thiết kế hệ thống SoC tích hợp AI và giới hạn về thời gian nghiên cứu, đồ án tập trung giải quyết các vấn đề cốt lõi với các giới hạn và phạm vi cụ thể như sau:

- **Phạm vi triển khai phần cứng:** Đề tài tập trung vào việc mô phỏng, tích hợp và triển khai thực nghiệm lõi vi xử lý mềm (Soft-core) RISC-V trên nền tảng FPGA Xilinx Kria KV260. Hệ thống dừng lại ở mức kiểm chứng kiến trúc trên nền tảng FPGA, không hướng tới việc sản xuất chip thương mại (ASIC).

- **Giới hạn về thiết kế vi kiến trúc:** Đề tài không đi sâu vào thiết kế lại chi tiết vi kiến trúc (Micro-architecture) của lõi CPU RISC-V ở mức RTL (Register Transfer Level) từ con số không. Thay vào đó, nhóm thực hiện tận dụng và tùy biến các nhân IP nguồn mở đã được kiểm chứng để tập trung nguồn lực vào việc thiết kế bộ tăng tốc AI (AI Accelerator) và hệ thống giao tiếp bus.

- **Phạm vi thuật toán và mô hình AI:** Việc xây dựng và huấn luyện mô hình (Training) được thực hiện Offline trên máy tính cá nhân sử dụng framework TensorFlow. Hệ thống phần cứng trên FPGA chỉ đảm nhận nhiệm vụ suy luận (Inference) dựa trên bộ trọng số đã được lượng tử hóa. Nhằm đảm bảo tính tối ưu của kiến trúc, đề tài chỉ tập trung gia tốc cho họ mạng Nơ-ron Tích chập (CNN - Convolutional Neural Network), không hỗ trợ các kiến trúc mạng hồi quy (RNN/LSTM) hoặc mô hình ngôn ngữ lớn (LLM/Transformer).

- **Phạm vi kịch bản ứng dụng (Use-case) và Dữ liệu:** Hệ thống được kiểm chứng thông qua các bài toán cốt lõi của Thị giác máy tính (Computer Vision) tại biên, bao gồm Phân loại ảnh (Image Classification) với các tập dữ liệu tiêu chuẩn (Fashion-MNIST/CIFAR-10) và Phát hiện đối tượng (Object Detection, cụ thể là Human Detection). Đề tài chưa mở rộng sang xử lý video thời gian thực độ phân giải cao (HD/Full HD) hay các loại tín hiệu âm thanh/văn bản.

- **Kiến trúc gia tốc và khả năng xử lý:** Bộ tăng tốc phần cứng được thiết kế với kiến trúc bộ đệm vòng (Circular Line Buffer) linh hoạt. Phần cứng hỗ trợ xử lý kích thước ảnh thay đổi tại thời gian thực (Runtime Configurable) thông qua cấu hình từ CPU ARM. Kích thước tối đa được hỗ trợ là 224x224 pixel (chuẩn của mạng VGG/ResNet) và hỗ trợ tương thích ngược với các độ phân giải thấp hơn (ví dụ

128x128 pixel) nhằm tối ưu điện năng tiêu thụ cho các ứng dụng Edge AI.

- **Đánh giá hiệu năng:** Việc đánh giá hệ thống được giới hạn ở các phép so sánh tương quan về thời gian xử lý (Inference Time), thông lượng (Throughput) và tài nguyên tiêu thụ (LUT, FF, BRAM, Power) giữa các kịch bản: chạy thuần phần mềm trên CPU ARM (có sẵn trên Kria KV260), chạy trên lõi RISC-V mềm, và chạy trên hệ thống RISC-V có tích hợp bộ tăng tốc phần cứng.

1

1.4. CẤU TRÚC BÁO CÁO

Báo cáo được tổ chức và trình bày theo trình tự logic gồm 5 chương, cụ thể như sau:

- **Chương 1. Giới thiệu đề tài:**
  Trình bày tổng quan về bối cảnh nghiên cứu, tính cấp thiết của việc phát triển Edge AI trên nền tảng phần cứng mở. Chương này cũng xác định rõ mục tiêu cốt lõi mà đề tài hướng tới, phạm vi nghiên cứu và tóm tắt bố cục của bài báo cáo.

- **Chương 2. Cơ sở lý thuyết và Các công trình liên quan:**
  Hệ thống hóa các kiến thức nền tảng cần thiết cho việc thực hiện đồ án, bao gồm: kiến trúc tập lệnh RISC-V, cấu trúc phần cứng FPGA (cụ thể là dòng Zynq UltraScale+), lý thuyết về Mạng nơ-ron tích chập (CNN) và chuẩn giao tiếp AXI4. Đồng thời, chương này cũng tổng hợp và phân tích các công trình nghiên cứu liên quan đã công bố để làm rõ vị trí và đóng góp của đề tài.

- **Chương 3. Đề xuất thiết kế hệ thống:**
  Mô tả chi tiết kiến trúc hệ thống SoC đề xuất, bao gồm sơ đồ khối tổng quát, phương án thiết kế các khối chức năng (IP Cores), chiến lược phân chia tác vụ giữa phần cứng/phần mềm (HW/SW Partitioning) và sơ đồ tổ chức bộ nhớ.

- **Chương 4. Hiện thực mô hình CNN đơn giản:**
  Trình bày quá trình xây dựng, huấn luyện và lượng tử hóa một mô hình CNN cơ bản (dựa trên tập dữ liệu Fashion-MNIST/CIFAR-10). Chương này cũng mô tả việc hiện thực mô hình tham chiếu để kiểm chứng giải thuật trước khi đưa xuống phần cứng.

- **Chương 5. Kết luận và Hướng phát triển:**
  Tổng kết các kết quả đã đạt được trong giai đoạn 1, phân tích những hạn chế và khó khăn kỹ thuật còn tồn tại. Từ đó, đề xuất lộ trình cụ thể cho giai đoạn 2, tập trung vào việc hoàn thiện tích hợp hệ thống và tối ưu hóa hiệu năng trên FPGA.

2

CƠ SỞ LÝ THUYẾT

2.1. KIẾN TRÚC TẬP LỆNH RISC-V

2.1.1. LỊCH SỬ HÌNH THÀNH

RISC-V là một kiến trúc tập lệnh (Instruction Set Architecture - ISA) dựa trên nguyên lý RISC (Reduced Instruction Set Computer), được đưa ra vào năm 2010 tại Phòng thí nghiệm Tính toán song song của Đại học California, Berkeley. Dự án được dẫn dắt bởi Giáo sư Krste Asanović cùng với sự tham gia của hai sinh viên cao học là Yunsup Lee và Andrew Waterman, dưới sự cố vấn của David Patterson – cha đẻ của kiến trúc RISC nguyên thủy.

Khác với các thế hệ RISC trước đó (như RISC-I, II, III, IV) vốn được thiết kế chủ yếu cho mục đích nghiên cứu học thuật, RISC-V ngay từ đầu đã được định hướng để trở thành một chuẩn ISA tiêu chuẩn cho cả nghiên cứu lẫn thương mại hóa. Mục tiêu của nhóm thiết kế là tạo ra một tập lệnh "sạch", không chịu gánh nặng của các vấn đề tương thích ngược từ các kiến trúc cũ kỹ tồn tại hàng thập kỷ, đồng thời hoàn toàn mở và miễn phí.

Năm 2015, RISC-V Foundation (nay là RISC-V International) được thành lập để quản lý chuẩn hóa và thúc đẩy hệ sinh thái phần cứng/phần mềm. Sự ra đời của RISC-V đã phá vỡ thế độc quyền lâu năm của các kiến trúc đóng như x86 (Intel) và ARM, mở ra kỷ nguyên "phần cứng mã nguồn mở"(Open Source Hardware).

2.1.2. CÁC ĐẶC ĐIỂM NỔI BẬT

So với các kiến trúc thương mại phổ biến, RISC-V sở hữu các đặc điểm kỹ thuật và pháp lý vượt trội:

- **Tính mở và Miễn phí (Open & Free):** RISC-V được phát hành dưới giấy phép BSD. Bất kỳ cá nhân hay tổ chức nào cũng có thể thiết kế, sản xuất và thương mại hóa chip RISC-V mà không phải trả phí bản quyền (Royalty-free). Điều này giúp giảm đáng kể chi phí khởi tạo cho các dự án khởi nghiệp và nghiên cứu.

- **Tính Mô đun:** Thay vì bắt buộc phải hiện thực toàn bộ tập lệnh khổng lồ, RISC-V có lõi là tập lệnh số nguyên cơ sở (Base Integer ISA) rất nhỏ gọn. Các tính năng nâng cao như số thực, nhân chia, hay nén lệnh (Compressed) được coi là các phần mở rộng tùy chọn. Điều này cho phép tối ưu hóa diện tích vi mạch cho các ứng dụng nhúng nhỏ gọn.

- **Khả năng mở rộng (Extensibility):** RISC-V dành riêng một phần không gian mã hóa lệnh (Opcode space) cho người dùng tự định nghĩa. Điều này đặc biệt quan trọng đối với các ứng dụng Edge AI, nơi các kỹ sư có thể thêm các lệnh tính toán ma trận hoặc tích chập chuyên biệt để tăng tốc phần cứng mà không vi phạm chuẩn ISA.

- **Kiến trúc Load/Store thuần túy:** Tương tự các kiến trúc RISC khác, RISC-V tách biệt hoàn toàn việc truy cập bộ nhớ và xử lý dữ liệu, giúp đơn giản hóa thiết kế phần cứng của Pipeline.

2.1.3. CẤU TRÚC TẬP LỆNH CƠ SỞ VÀ CÁC PHẦN MỞ RỘNG

TẬP LỆNH SỐ NGUYÊN CƠ SỞ (BASE INTEGER ISA)

Nền tảng của RISC-V là tập lệnh số nguyên cơ sở, được ký hiệu là **RV32I** (cho bản 32-bit), RV64I (64-bit) hoặc RV128I (128-bit). Trong phạm vi đồ án này, nhóm tập trung vào **RV32I**, bao gồm 47 lệnh cơ bản.

Hệ thống thanh ghi (Register File): RV32I cung cấp 32 thanh ghi đa năng (General Purpose Registers - GPR), mỗi thanh ghi rộng 32-bit, được đánh số từ $x0$ đến $x31$. Ngoài ra còn có một thanh ghi đếm chương trình (Program Counter - PC).

Theo quy ước gọi hàm (Calling Convention) của RISC-V ABI, các thanh ghi này có vai trò cụ thể như mô tả trong Bảng 2.1:

Tên	ABI Name	Mô tả	Lưu bởi
x0	zero	Luôn có giá trị bằng 0 (Hardwired Zero)	-
x1	ra	Địa chỉ trả về (Return Address)	Caller
x2	sp	Con trỏ ngăn xếp (Stack Pointer)	Callee
x3	gp	Con trỏ toàn cục (Global Pointer)	-
x4	tp	Con trỏ luồng (Thread Pointer)	-
x5-x7	t0-t2	Thanh ghi tạm thời (Temporaries)	Caller
x8	s0/fp	Thanh ghi lưu trữ/Con trỏ khung (Frame Ptr)	Callee
x9	s1	Thanh ghi lưu trữ (Saved Register)	Callee
x10-x11	a0-a1	Tham số hàm / Giá trị trả về	Caller
x12-x17	a2-a7	Tham số hàm (Function Arguments)	Caller
x18-x27	s2-s11	Thanh ghi lưu trữ (Saved Registers)	Callee
x28-x31	t3-t6	Thanh ghi tạm thời (Temporaries)	Caller

Bảng 2.1: Quy ước sử dụng thanh ghi trong RISC-V (ABI)

ĐỊNH DẠNG LỆNH (INSTRUCTION FORMATS)

Để đơn giản hóa bộ giải mã, các lệnh trong RV32I có độ dài cố định 32-bit và được căn chỉnh bộ nhớ (aligned) theo địa chỉ chia hết cho 4. RISC-V định nghĩa 6 định dạng lệnh cơ bản:

- **R-type (Register):** Dùng cho các phép toán giữa các thanh ghi (VD: cộng, trừ, logic). Cấu trúc gồm: opcode, rd (đích), funct3, rs1 (nguồn 1), rs2 (nguồn 2), funct7.
- **I-type (Immediate):** Dùng cho các phép toán với hằng số ngắn hoặc lệnh Load. Gồm: opcode, rd, funct3, rs1, imm[11:0].
- **S-type (Store):** Dùng cho lệnh lưu dữ liệu ra bộ nhớ (Store). Hằng số immediate được chia làm 2 phần để giữ vị trí các thanh ghi nguồn cố định.
- **B-type (Branch):** Dùng cho các lệnh rẽ nhánh có điều kiện. Tương tự S-type nhưng immediate được mã hóa khác để mở rộng phạm vi nhảy.
- **U-type (Upper Immediate):** Dùng cho lệnh thao tác với 20-bit cao của thanh ghi (VD: LUI, AUIPC).
- **J-type (Jump):** Dùng cho lệnh nhảy không điều kiện (JAL).

Các chế độ đặc quyền

RISC-V định nghĩa ba chế độ đặc quyền để bảo vệ hệ thống:

1. **Machine Mode (M-Mode):** Chế độ có quyền cao nhất, truy cập toàn bộ tài nguyên phần cứng. Các vi điều khiển đơn giản (bare-metal) chỉ cần chạy ở chế độ này.

2. **Supervisor Mode (S-Mode):** Dùng cho hệ điều hành (như Linux, FreeRTOS) để quản lý bộ nhớ ảo và bảo vệ tài nguyên.

3. **User Mode (U-Mode):** Chế độ có quyền thấp nhất, dành cho các ứng dụng người dùng.

Trong đồ án này, lõi RISC-V chủ yếu sẽ hoạt động ở M-Mode để tương tác trực tiếp với phần cứng FPGA và bộ tăng tốc.

2.1.4. So sánh kiến trúc RISC và CISC

Để làm rõ ưu thế của việc lựa chọn RISC-V cho đề tài, cần đặt nó vào hệ quy chiếu so sánh với kiến trúc CISC (Complex Instruction Set Computer) – triết lý thiết kế đối lập vốn thống trị thị trường máy tính cá nhân (PC) nhiều thập kỷ qua.

Triết lý thiết kế

Sự khác biệt căn bản giữa RISC và CISC nằm ở cách phân chia khối lượng công việc giữa phần cứng (Hardware) và phần mềm (Software/Compiler):

- **CISC:** Tập trung vào việc giảm "khoảng cách ngữ nghĩa"(semantic gap) giữa ngôn ngữ bậc cao và mã máy. CISC cung cấp các lệnh phức tạp, đa năng (ví dụ: một lệnh duy nhất có thể thực hiện nạp dữ liệu, cộng và lưu kết quả). Điều này giúp giảm kích thước mã chương trình (Code size) nhưng làm tăng độ phức tạp của phần cứng giải mã lệnh.

- **RISC:** Chuyển gánh nặng tối ưu hóa sang trình biên dịch (Compiler). Tập lệnh chỉ bao gồm các chỉ thị đơn giản, thực thi nhanh. Các tác vụ phức tạp được thực hiện bằng cách kết hợp nhiều lệnh đơn giản. Điều này giúp phần cứng đơn giản hơn, tiết kiệm diện tích silicon để dành cho bộ nhớ đệm (Cache) hoặc các lõi xử lý song song.

So sánh RISC và CISC

Bảng 2.2 tóm tắt các đặc điểm kỹ thuật đối lập giữa hai kiến trúc [1]:

Tiêu chí	RISC (RISC-V, ARM)	CISC (x86)
Độ phức tạp lệnh	Đơn giản, thực hiện tác vụ nhỏ.	Phức tạp, một lệnh làm việc (truy cập bộ nhớ toán).
Thời gian thực thi	Hướng tới 1 chu kỳ máy/lệnh (CPI ≈ 1).	Đa chu kỳ (Multi-cycle), kỳ thay đổi tùy lệnh.
Định dạng lệnh	Cố định (thường là 32-bit), giải mã nhanh.	Thay đổi (Variable length mã phức tạp.
Truy cập bộ nhớ	Chỉ qua lệnh Load/Store.	Các lệnh tính toán có thể tác trực tiếp với bộ nhớ.
Số lượng thanh ghi	Nhiều (32 hoặc hơn) để giảm truy cập RAM.	Ít thanh ghi chuyên dụng
Pipiline	Dễ dàng hiện thực và đạt hiệu quả cao.	Khó hiện thực do độ dà không đồng nhất.
Kích thước chương trình	Lớn hơn (cần nhiều lệnh để làm một việc).	Nhỏ gọn hơn (mật độ mã

Bảng 2.2: So sánh đặc điểm kỹ thuật giữa RISC và CISC

2.1.5. CÁC PHẦN MỞ RỘNG TIÊU CHUẨN (STANDARD EXTENSIONS)

Để tăng cường khả năng xử lý, RISC-V cung cấp các phần mở rộng được chuẩn hóa. Một vi xử lý hỗ trợ đầy đủ các mở rộng cơ bản thường được ký hiệu là **RV32G** (General), trong đó G = I + M + A + F + D.

- **M (Multiplication/Division):** Bổ sung các lệnh nhân và chia số nguyên phần cứng. Nếu không có mở rộng này, CPU phải thực hiện nhân/chia bằng phần mềm rất chậm. Đây là mở rộng bắt buộc cho các ứng dụng AI.
- **A (Atomic):** Cung cấp các lệnh truy cập bộ nhớ nguyên tử (Read-Modify-Write), cần thiết để hỗ trợ hệ điều hành đa nhiệm (như Linux) và đồng bộ hóa đa luồng.
- **F (Single-Precision Floating Point):** Hỗ trợ tính toán số thực dấu chấm động 32-bit (chuẩn IEEE 754-2008), bổ sung thêm 32 thanh ghi số thực $f0 - f31$.
- **D (Double-Precision Floating Point):** Hỗ trợ số thực 64-bit.
- **C (Compressed Instructions):** Hỗ trợ các lệnh nén độ dài 16-bit.

Mở rộng này giúp giảm kích thước chương trình (code size) từ 25-30%, cực kỳ hữu ích cho các thiết bị nhúng có bộ nhớ hạn chế.

- **V (Vector Operations):** Mở rộng xử lý vector, cho phép thực hiện một lệnh trên nhiều dữ liệu (SIMD). Đây là mở rộng quan trọng nhất đối với các ứng dụng học sâu (Deep Learning) và xử lý ảnh hiện đại, tuy nhiên trong phạm vi đồ án này, chúng ta sẽ thiết kế bộ tăng tốc riêng thay vì dùng V-extension tiêu chuẩn.

2.1.6. Sự phù hợp của RISC đối với Edge AI

Mặc dù CISC vẫn thống trị mảng PC/Server nhờ tính tương thích ngược, RISC (đặc biệt là RISC-V) lại chiếm ưu thế tuyệt đối trong lĩnh vực Hệ thống nhúng và IoT vì các lý do sau:

1. **Hiệu quả năng lượng (Power Efficiency):** Do phần cứng giải mã đơn giản, các chip RISC tiêu thụ ít điện năng hơn đáng kể, phù hợp cho các thiết bị chạy pin.
2. **Diện tích nhỏ:** Thiết kế RISC chiếm ít tài nguyên LUTs/Flip-flops hơn khi triển khai trên FPGA, để dành không gian cho các bộ tăng tốc AI.
3. **Dễ dàng tùy biến:** Cấu trúc lệnh đơn giản của RISC giúp các nhà thiết kế dễ dàng thêm các lệnh tùy chỉnh (Custom Instructions) cho AI mà không làm phá vỡ kiến trúc đường ống, điều rất khó thực hiện trên CISC.

2.2. Lõi vi xử lý CV32E40P

Sau khi đã trình bày chuẩn kiến trúc tập lệnh (ISA) RISC-V ở mục 2.1, phần này tập trung vào hiện thực phần cứng cụ thể được sử dụng trong đồ án: lõi vi xử lý mã nguồn mở **CV32E40P**. Đây là lựa chọn nền tảng cho phân hệ điều khiển bên trong vùng logic khả trình (PL) của FPGA Kria KV260.

2.2.1. Tổng quan và lý do lựa chọn

CV32E40P (tên mã cũ *RI5CY*) là một lõi RISC-V 32-bit do nhóm nghiên cứu PULP (Parallel Ultra Low Power) thuộc ETH Zurich và Đại học Bologna phát triển, hiện được chuẩn hóa và bảo trì bởi tổ chức OpenHW Group. Lõi này tuân thủ chuẩn RISC-V với tập lệnh cơ sở RV32I và hỗ trợ các phần mở rộng M (Multiply/Divide) và C (Compressed), đáp ứng đầy đủ yêu cầu cho một vi điều khiển hệ thống nhúng.

Trong số nhiều lõi RISC-V mã nguồn mở (Ibex, VexRiscv, Rocket, BOOM,...), CV32E40P được lựa chọn cho đồ án vì các lý do sau:

- **Diện tích nhỏ gọn:** Đường ống 4 tầng cho phép tổng hợp với số lượng LUT/FF tối thiểu, dành chỗ cho bộ tăng tốc AI và các IP khác trên FPGA.
- **Cộng đồng và tài liệu:** Là lõi tiêu chuẩn của OpenHW Group, có RTL đã được kiểm chứng công nghiệp, tài liệu Verification Plan và testbench đầy đủ.
- **Khả năng tùy biến:** Mã nguồn SystemVerilog rõ ràng, dễ tinh chỉnh để loại bỏ các khối không sử dụng (FPU, các tập lệnh mở rộng PULP) nhằm tối ưu tài nguyên.
- **Giao tiếp OBI tiêu chuẩn:** Sử dụng *Open Bus Interface*, dễ chuyển đổi sang AXI4 thông qua cầu nối (bridge) để tích hợp vào hệ sinh thái Vivado.

Trong đồ án, CV32E40P được cấu hình với các tham số rút gọn nhằm tối ưu cho vai trò *điều phối luồng thực thi* (orchestrator) cho bộ tăng tốc CNN, không trực tiếp tham gia tính toán nặng:

- ISA: RV32IMC (số nguyên + nhân/chia + lệnh nén).
- FPU: **Disabled** – toàn bộ mô hình AI đã được lượng tử hóa INT8.
- Tập lệnh mở rộng PULP (Xpulp): **Disabled** – nhằm chuẩn hóa tuân thủ RISC-V và giảm độ phức tạp logic.

2.2.2. CẤU TRÚC ĐƯỜNG ỐNG LỆNH

Đường ống thực thi của CV32E40P bao gồm 4 giai đoạn được mô tả như sau:

1. **Instruction Fetch (IF):** Nạp lệnh từ bộ nhớ chương trình (I-BRAM trên FPGA, ánh xạ tại địa chỉ `0x0000_0000`). Khối *prefetch buffer* cho phép nạp trước nhiều lệnh để giảm độ trễ rẽ nhánh.
2. **Instruction Decode (ID):** Giải mã trường opcode, đọc các toán hạng từ tập thanh ghi 32 thanh ghi đa năng. Tại đây bộ điều khiển nhánh (branch resolution) cũng được kích hoạt sớm để giảm penalty.
3. **Execute (EX):** Khối ALU thực thi các phép toán số nguyên cơ bản; bộ nhân/chia hoạt động trong nhiều chu kỳ. Các lệnh truy cập bộ nhớ phát yêu cầu OBI ra giao tiếp `m_axi_data`.

4. **Write Back (WB):** Ghi kết quả tính toán hoặc dữ liệu đọc bộ nhớ trở lại tập thanh ghi.

Đường ống ngắn (4 tầng) của CV32E40P giúp giảm thiểu chi phí phạt (penalty) khi dự đoán sai nhánh và đơn giản hóa phần cứng so với các kiến trúc đường ống dài (như Rocket 5-tầng hay BOOM 10-tầng). Cấu hình này phù hợp với mục tiêu *xử lý điều khiển* – chủ yếu là đọc/ghi thanh ghi cấu hình, kiểm tra cờ trạng thái, và điều phối DMA – thay vì tính toán cường độ cao.

2.2.3. GIAO TIẾP OPEN BUS INTERFACE (OBI)

CV32E40P xuất hai cổng OBI độc lập theo kiến trúc Harvard:

- **Instruction OBI port**: nạp lệnh, chỉ đọc.
- **Data OBI port**: đọc/ghi dữ liệu, truy cập ngoại vi và bộ nhớ chia sẻ.

OBI là một giao thức bộ nhớ đơn giản 1 chiều, sử dụng cặp tín hiệu bắt tay `req/gnt` cho địa chỉ và `rvalid` cho dữ liệu trả về. Tuy nhiên, môi trường Vivado và phần cứng Zynq UltraScale+ sử dụng chuẩn AMBA AXI4 làm giao thức xương sống. Do đó, đề tài xây dựng cầu nối `obi_to_axi` để chuyển đổi cả hai cổng OBI sang AXI4-Lite, cho phép tích hợp lõi RISC-V vào Block Design Vivado đồng nhất với các IP khác. Chi tiết kỹ thuật của cầu nối này được trình bày tại mục **??**.

2.3. NỀN TẢNG PHẦN CỨNG FPGA VÀ SYSTEM-ON-CHIP

Để hiện thực hóa một hệ thống Edge AI có khả năng tùy biến cao, việc lựa chọn nền tảng phần cứng là yếu tố then chốt. Phần này trình bày từ các khái niệm cơ bản về công nghệ FPGA, kiến trúc hệ thống trên vi mạch (SoC) đến chi tiết kỹ thuật của kit phát triển Kria KV260 được sử dụng trong đồ án.

2.3.1. CÔNG NGHỆ FPGA (FIELD PROGRAMMABLE GATE ARRAY)

KHÁI NIỆM VÀ KIẾN TRÚC CƠ BẢN

FPGA (Field Programmable Gate Array - Mảng cổng lập trình dạng trường) là một loại vi mạch bán dẫn đặc biệt cho phép người dùng tái cấu hình phần cứng sau khi đã sản xuất. Khác với ASIC (Application Specific Integrated Circuit) vốn có chức năng cố định vĩnh viễn, FPGA bao gồm một ma trận các khối logic khả trình (Configurable Logic Blocks - CLBs) được kết nối với nhau thông qua mạng lưới dây dẫn có thể lập trình (Programmable Interconnects).[2]

Thành phần cấu tạo cơ bản của FPGA bao gồm:

- **Look-Up Table (LUT):** Đơn vị cơ bản nhất dùng để hiện thực các hàm logic tổ hợp (Boolean logic). Một LUT N-đầu vào có thể coi như một bộ nhớ RAM nhỏ lưu trữ bảng chân trị của hàm số.
- **Flip-Flops (FF):** Phần tử nhớ dùng để lưu trạng thái, kết hợp với LUT để tạo thành các mạch tuần tự (Sequential Logic).
- **DSP Slices (Digital Signal Processing):** Các khối phần cứng chuyên dụng thực hiện phép nhân và cộng (Multiply-Accumulate - MAC) tốc độ cao. Đây là tài nguyên quan trọng nhất đối với các ứng dụng xử lý tín hiệu số và Trí tuệ nhân tạo.
- **Block RAM (BRAM):** Các khối bộ nhớ tĩnh (SRAM) nằm rải rác trong chip, cung cấp bộ nhớ đệm cục bộ với băng thông cực lớn và độ trễ thấp.

ƯU ĐIỂM TRONG ỨNG DỤNG AI

Nhờ khả năng xử lý song song ở mức phần cứng, FPGA có thể thực hiện hàng nghìn phép tính nhân-cộng đồng thời, phù hợp với đặc thù tính toán ma trận dày đặc của mạng CNN. Hơn nữa, khả năng tùy biến luồng dữ liệu (Dataflow) giúp loại bỏ các nút thắt cổ chai về bộ nhớ thường gặp trên các kiến trúc Von Neumann truyền thống.

2.3.2. SYSTEM ON CHIP - SOC

Trong các thiết kế hiện đại, FPGA hiếm khi đứng độc lập mà thường được tích hợp trong một Hệ thống trên vi mạch (SoC). Kiến trúc này, thường được gọi là SoC FPGA hoặc MPSoC (Multi-Processor SoC), kết hợp hai thế giới:

1. **Hệ thống xử lý (Processing System - PS):** Bao gồm các nhân vi xử lý cứng (Hard-core) như ARM Cortex-A, chạy hệ điều hành (Linux/RTOS) để quản lý kết nối mạng, giao diện người dùng và các tác vụ cấp cao.

2. **Phần logic lập trình được (Programmable Logic - PL):** Chính là phần FPGA, dùng để thực thi các tác vụ tính toán nặng, xử lý tín hiệu thời gian thực hoặc hiện thực các giao tiếp tùy chỉnh.

Sự kết hợp này cho phép thực hiện đồng thiết kế phần cứng/phần mềm (Hardware/Software Co-design). Trong đồ án này, phần PL sẽ được sử dụng để nhúng lõi RISC-V mềm (Soft-core) và bộ tăng tốc AI, trong khi phần PS đóng vai trò giám sát và nạp chương trình.

2.3.3. NỀN TẢNG KRIA KV260 VISION AI

Để triển khai thực nghiệm, đồ án sử dụng bộ công cụ **Kria KV260 Vision AI Starter Kit** của hãng Xilinx. Đây là nền tảng phần cứng được tối ưu hóa đặc biệt cho các ứng dụng thị giác máy tính tại biên.

KIẾN TRÚC SYSTEM ON MODULE

Điểm khác biệt của dòng Kria so với các kit FPGA truyền thống là việc áp dụng kiến trúc SOM. Kit KV260 bao gồm hai phần tách biệt:

- **K26 SOM:** Mô-đun tính toán trung tâm kích thước nhỏ gọn, chứa chip xử lý chính, bộ nhớ RAM (4GB DDR4), bộ nhớ Flash (16GB eMMC) và các mạch quản lý nguồn (PMIC).

- **Carrier Card:** Bo mạch đế cung cấp các giao tiếp ngoại vi như Ethernet, USB 3.0, HDMI, DisplayPort và các cổng kết nối Camera (MIPI CSI-2).

CHIP ZYNQ ULTRASCALE+ MPSOC

Trái tim của K26 SOM là chip **XCK26 Zynq UltraScale+ MPSoC**. Đây là dòng chip cao cấp được sản xuất trên tiến trình 16nm FinFET+, cung cấp tài nguyên dồi dào cho thiết kế:

- **Processing System (PS):**

  - Quad-core ARM Cortex-A53 (lên đến 1.5GHz) cho ứng dụng hệ thống.
  - Dual-core ARM Cortex-R5F cho xử lý thời gian thực.
  - GPU Mali-400 MP2 cho xử lý đồ họa.

- **Programmable Logic (PL):**
  - **Logic Cells:** 256,200 ô logic (tương đương khoảng 1.2 triệu cổng ASIC) - Đủ lớn để chứa lõi RISC-V phức tạp.
  - **DSP Slices:** 1,248 khối DSP48E2 - Cung cấp năng lực tính toán lên tới hàng nghìn tỷ phép tính mỗi giây (TOPS) cho AI.
  - **Block RAM:** 144 khối (36Kb mỗi khối) và 64 khối UltraRAM, cung cấp băng thông nội bộ cực lớn.

2.3.4. Nguồn dữ liệu đầu vào cho hệ thống

Trong phạm vi đồ án này, hệ thống Edge-AI thực hiện phân loại nhị phân (binary classification) trên **ảnh tĩnh đọc từ thẻ nhớ SD**, không tích hợp luồng video thời gian thực từ Camera. Quyết định này phù hợp với mục tiêu kiểm chứng kiến trúc đồng thiết kế phần cứng-phần mềm: bộ tăng tốc Conv-CNN, lõi RISC-V điều phối, và runtime ARM PYNQ. Việc tích hợp pipeline thu nhận ảnh real-time qua MIPI CSI-2 + ISP + VDMA được dành cho hướng phát triển tiếp theo (xem chương **??**).

Trên KV260, ảnh được vi xử lý ARM (chạy hệ điều hành Ubuntu + PYNQ) đọc trực tiếp từ thẻ nhớ thông qua các thư viện Python tiêu chuẩn (`cv2.imread`). Sau khi tiền xử lý (resize về kích thước 128×128, chuẩn hóa và lượng tử hóa INT8), ảnh được nạp vào bộ nhớ DDR thông qua cấp phát CMA (`pynq.allocate`). Lõi RISC-V và bộ tăng tốc CNN truy cập vùng DDR này thông qua bus AXI4 cache-coherent (cổng `S_AXI_HPC0_FPD`). Chi tiết luồng dữ liệu được trình bày tại chương 4.

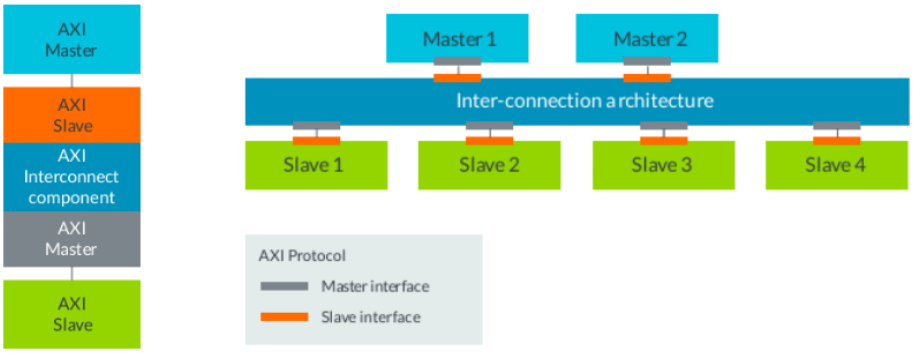
2.4. Tổng quan chuẩn giao tiếp AXI

2.4.1. Lịch sử hình thành và sự hội tụ giữa ASIC và FPGA

AXI (Advanced eXtensible Interface) là một giao thức xương sống thuộc bộ tiêu chuẩn AMBA (Advanced Microcontroller Bus Architecture) do ARM giới thiệu lần đầu vào năm 1996. Ban đầu, AXI được thiết kế chuyên biệt làm "ngôn ngữ giao tiếp"chuẩn mực trong thế giới vi mạch chuyên dụng (ASIC - Application-Specific Integrated Circuit). Trong các hệ thống SoC (System-on-Chip) phức tạp của ASIC, sự hiện diện của hàng chục khối sở hữu trí tuệ (IP Cores) khác nhau như CPU, GPU, bộ giải mã video,

và bộ điều khiển bộ nhớ đòi hỏi một chuẩn bus giao tiếp chung. AXI ra đời giải quyết bài toán tái sử dụng lõi IP (IP Reuse), cho phép các nhà thiết kế ghép nối các module từ nhiều nhà cung cấp khác nhau một cách liền mạch mà không cần thiết kế lại giao diện vật lý.

Nhận thấy ưu điểm vượt trội về băng thông và tính linh hoạt của AMBA AXI, từ thế hệ Spartan-6 và Virtex-6, Xilinx (nay là AMD) đã bắt đầu loại bỏ các chuẩn bus cũ (như CoreConnect, PLB) để chuyển sang áp dụng AXI. Đặc biệt, với sự ra đời của họ Zynq-7000 và Zynq UltraScale+, Xilinx đã biến AXI4 (thế hệ thứ 4 của kiến trúc AMBA) thành tiêu chuẩn giao tiếp duy nhất để làm cầu nối giữa Hệ thống xử lý đa nhân (PS - Processing System) và Vùng logic khả trình (PL - Programmable Logic). Sự hội tụ này đã xóa nhòa ranh giới giữa ASIC và FPGA, biến FPGA thành một nền tảng tạo mẫu nhanh (ASIC Prototyping) hoàn hảo, nơi các kiến trúc phần cứng có thể được kiểm chứng thực tế trước khi tiến hành sản xuất hàng loạt (Tape-out).

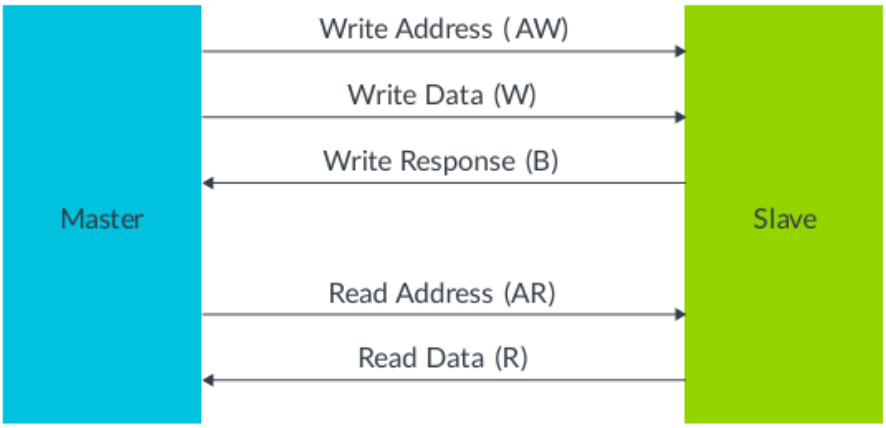


Hình 2.1: Sơ đồ AXI (AMBA)

2.4.2. Cơ sở lý thuyết và Cơ chế hoạt động của AXI4

Giao thức AXI4 hoạt động dựa trên cơ chế ánh xạ bộ nhớ (Memory-Mapped I/O). Trong kiến trúc này, vi xử lý trung tâm không giao tiếp với các thiết bị ngoại vi thông qua các lệnh I/O đặc biệt, mà coi toàn bộ các khối phần cứng (Slave) như những dải địa chỉ bộ nhớ (Memory Addresses) liền kề [3].

Để đảm bảo hiệu suất truyền tải cao và hỗ trợ thực thi không theo thứ tự (Out-of-order execution), kiến trúc AXI4 phân tách các luồng dữ liệu thành 5 kênh (channels) độc lập, hoạt động song song và truyền theo một chiều duy nhất:



Hình 2.2: Các kênh giao tiếp (AMBA)

1. **Kênh Địa chỉ Đọc (Read Address Channel - AR):** Truyền tải địa chỉ bắt đầu và các thông tin điều khiển cho quá trình đọc.
2. **Kênh Dữ liệu Đọc (Read Data Channel - R):** Truyền dữ liệu từ Slave về Master, kèm theo cờ báo trạng thái đọc.
3. **Kênh Địa chỉ Ghi (Write Address Channel - AW):** Truyền tải địa chỉ bắt đầu cho quá trình ghi dữ liệu.
4. **Kênh Dữ liệu Ghi (Write Data Channel - W):** Truyền dữ liệu thực tế từ Master xuống Slave.
5. **Kênh Phản hồi Ghi (Write Response Channel - B):** Slave phản hồi lại Master về trạng thái của toàn bộ giao dịch ghi (thành công hoặc có lỗi).

Cơ chế bắt tay: Điểm cốt lõi làm nên độ tin cậy của AXI là cơ chế bắt tay hai chiều thông qua cặp tín hiệu `VALID` và `READY` có mặt trên tất cả 5 kênh. Master kéo tín hiệu `VALID` lên mức logic cao (1) để thông báo rằng dữ liệu hoặc địa chỉ trên bus đã hợp lệ. Ngược lại, Slave báo hiệu khả năng tiếp nhận bằng cách kéo tín hiệu `READY` lên cao. Một gói dữ liệu chỉ được coi là truyền thành công khi và chỉ khi cả `VALID` và `READY` đều ở mức 1 tại thời điểm sườn lên của xung nhịp (Clock Edge). Cơ chế này cho phép các khối phần cứng hoạt động ở các tần số xung nhịp khác nhau có thể đồng bộ hóa mà không gây ra hiện tượng mất mát dữ liệu hay thắt cổ chai (bottleneck).

2.4.3. PHÂN LOẠI CÁC BIẾN THỂ AXI TRÊN NỀN TẢNG XILINX

Theo tài liệu *Vitis Model Composer User Guide (UG1483)* của AMD/Xilinx **ug1483**, giao thức AXI là tiêu chuẩn xương sống để đóng gói và kết nối các khối phần cứng. Tài liệu quy định quy trình ánh xạ (mapping) tự động các tín hiệu thuật toán cấp cao xuống ba biến thể AXI vật lý chính nhằm tối ưu hóa tài nguyên phần cứng và băng thông trên nền tảng FPGA:

- **AXI4-Lite (Giao tiếp điều khiển vô hướng - Scalar):** Theo UG1483, AXI4-Lite được sử dụng mặc định để ánh xạ các cổng tín hiệu đầu vào/đầu ra dạng vô hướng và các tín hiệu điều khiển (Control signals). Giao diện này tạo ra một bảng địa chỉ thanh ghi (Memory-Mapped Registers). Với đặc tính chỉ hỗ trợ truyền một đơn vị dữ liệu trong mỗi giao dịch (Single beat) và thiết kế logic cực kỳ nhỏ gọn, AXI4-Lite phù hợp để vi xử lý đọc/ghi trực tiếp vào các thanh ghi tĩnh của mạch gia tốc. Các thao tác điển hình bao gồm: ghi bit 1 để khởi động mạch (Start), đọc bit cờ báo hoàn thành (Done/Idle), hoặc nạp các tham số siêu tham số (Hyper-parameters) cho thuật toán.

- **AXI4 Memory-Mapped (AXI4-Full - Truy xuất khối tốc độ cao):** Biến thể này được ứng dụng khi mô hình thiết kế cần giao tiếp với bộ nhớ đệm dùng chung dung lượng lớn như DDR RAM hoặc Block RAM (BRAM). Sức mạnh của AXI4-Full nằm ở khả năng truyền dữ liệu theo lô (Burst Transfer). CPU hoặc bộ điều khiển DMA (Direct Memory Access) chỉ cần cung cấp địa chỉ bắt đầu một lần duy nhất, sau đó có thể vận chuyển hàng ngàn byte dữ liệu (ví dụ: các bản đồ đặc trưng - Feature Maps của mạng CNN hoặc trọng số Weight) trong một chu kỳ thiết lập. Tính năng này giải quyết triệt để vấn đề "Bức tường bộ nhớ"(Memory Wall) trong các thiết kế gia tốc phần cứng AI.

- **AXI4-Stream (Dữ liệu luồng hướng sự kiện):** Tài liệu chỉ định AXI4-Stream cho các luồng dữ liệu mảng (Array) tốc độ cực cao, phổ biến trong Xử lý tín hiệu số (DSP) và Xử lý ảnh (Vision AI). Khác biệt hoàn toàn với hai biến thể trên, AXI4-Stream truyền dữ liệu mà không cần địa chỉ bộ nhớ (Address-less). Nó hoạt động như một đường ống nước (Pipeline) kết nối trực tiếp đầu ra của khối này vào đầu vào của khối kia. Giao thức này chỉ sử dụng cơ chế bắt tay `TVALID` và `TREADY` để đẩy thông lượng (Throughput) lên mức tối đa mà không bị giới hạn bởi chu kỳ giải mã địa chỉ.

2.4.4. TÍCH HỢP HỆ THỐNG

Quy trình đóng gói phần cứng (IP Packaging): Một điểm nhấn quan trọng trong hệ sinh thái thiết kế của Xilinx (UG1483) là khả năng tự động hóa việc đóng gói. Khi thiết kế lõi thuật toán bằng ngôn ngữ mô tả phần cứng (Verilog/SystemVerilog), công cụ IP Packager không chỉ tổng hợp mạch logic lõi mà còn tự động sinh ra lớp bọc (wrapper) chứa các giao diện AXI chuẩn mực. Đồng thời, hệ thống cũng tự động khởi tạo bộ thư viện trình điều khiển phần mềm (C/C++ Driver API / BSP). Điều này cho phép hệ điều hành hoặc mã Bare-metal dễ dàng tương tác với khối phần cứng thông qua các hàm truy xuất thanh ghi tiêu chuẩn trong môi trường Vitis thống nhất.

Ứng dụng chuẩn AXI vào Kiến trúc Gia tốc Mạng nơ-ron: Dựa trên nền tảng lý thuyết của giao thức AXI, kiến trúc của đồ án được định hình xoay quanh việc triển khai hệ thống nhận diện vật thể ứng dụng mô hình CNN lượng tử hóa (INT8) trên SoC Zynq UltraScale+ (Kit phát triển Kria KV260). Hệ thống là sự kết hợp đồng bộ giữa phần mềm chạy trên tập lệnh RISC-V (lõi mềm CV32E40P) và phần cứng gia tốc logic (CNN Accelerator). Giao thức AXI đóng vai trò cốt lõi trong việc định tuyến toàn bộ luồng điều khiển và luồng dữ liệu của thiết kế:

- **Luồng Điều khiển (Control Flow) với AXI4-Lite:** Khối gia tốc phần cứng được tích hợp cổng giao tiếp Slave AXI4-Lite, được ánh xạ vào một dải địa chỉ cơ sở tĩnh trên không gian bộ nhớ của hệ thống (Memory-mapped). Vi xử lý trung tâm sử dụng đường truyền này để can thiệp trực tiếp vào máy trạng thái (FSM) của khối CNN. Cụ thể, các lệnh ghi/đọc được sử dụng để kích hoạt cờ vận hành (Start), hoặc kiểm tra trạng thái hoạt động (Status/Done). Để tối ưu hóa thời gian tính toán song song, hệ thống được thiết kế kết hợp phát tín hiệu ngắt cứng (IRQ) qua bộ điều khiển ngắt, giải phóng CPU khỏi việc phải liên tục chờ đợi vòng lặp (Polling).

- **Luồng Dữ liệu (Data Flow) với Kiến trúc Bộ nhớ dùng chung (Shared Memory):** Để giải quyết bài toán thắt cổ chai dữ liệu khi cả lõi RISC-V và lõi gia tốc CNN cùng cần truy xuất thông tin, hệ thống sử dụng một khối Block RAM (BRAM) cấu hình cổng kép (True Dual-Port BRAM) làm vùng nhớ dùng chung. Giao diện AXI4 kết nối vùng vi xử lý Host (nhân Cortex-A) và lõi RISC-V vào hệ thống BRAM này. Ban đầu, vi xử lý Host đóng vai trò quản lý tài nguyên, thực hiện nạp firmware cho RISC-V và đưa dữ liệu ảnh đầu vào (Inference Data) xuống BRAM thông qua bus AXI. Sau khi giải phóng tín hiệu Reset,

lõi RISC-V sẽ trực tiếp nạp lệnh từ BRAM và đóng vai trò điều khiển độc lập để chỉ đạo khối CNN tính toán.

Việc tuân thủ chặt chẽ tiêu chuẩn giao tiếp AMBA AXI trong thiết kế này không chỉ đảm bảo hệ thống vận hành ổn định trên nền tảng FPGA Kria KV260, mà còn mang ý nghĩa lớn về mặt kiến trúc. Nền tảng Zynq hiện tại đóng vai trò là môi trường ASIC Prototyping. Khi định hướng đưa hệ thống vi mạch Edge AI này vào sản xuất thực tế (Tape-out), toàn bộ lõi RISC-V và bộ gia tốc CNN chuẩn AXI có thể được tái sử dụng trực tiếp trên đế Silicon (ASIC) mà không cần phải tái cấu trúc giao thức giao tiếp, khẳng định tính chuyên nghiệp và định hướng công nghiệp của đồ án.

2.5. TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO (AI)

2.5.1. CÁC MÔ HÌNH AI (AI MODELS)

Trong lĩnh vực thị giác máy tính và hệ thống nhúng, các mô hình AI thường được phân loại dựa trên chức năng và kiến trúc. Việc lựa chọn mô hình phù hợp là yếu tố quyết định đến hiệu năng của hệ thống Edge AI.

MÔ HÌNH PHÂN LOẠI (CLASSIFICATION)

Đây là dạng mô hình cơ bản nhất, nhiệm vụ là gán nhãn cho một hình ảnh đầu vào. Ví dụ: xác định xem trong ảnh có người hay không. Các kiến trúc tiêu biểu bao gồm MobileNet, SqueezeNet, và ResNet. Đối với các thiết bị biên, MobileNet thường được ưu tiên nhờ kiến trúc Depthwise Separable Convolution giúp giảm đáng kể khối lượng tính toán.

MÔ HÌNH PHÁT HIỆN ĐỐI TƯỢNG (OBJECT DETECTION)

Phức tạp hơn phân loại, mô hình này không chỉ xác định đối tượng là gì mà còn định vị vị trí của nó thông qua các khung bao (Bounding Box). Các đại diện nổi bật gồm dòng YOLO (You Only Look Once) và SSD (Single Shot MultiBox Detector). Tuy nhiên, việc triển khai Object Detection trên soft-core RISC-V đòi hỏi tài nguyên tính toán rất lớn.

MÔ HÌNH PHÂN ĐOẠN (SEGMENTATION)

Phân loại từng pixel trong ảnh, giúp tách nền và đối tượng một cách chi tiết. Dạng mô hình này (ví dụ: U-Net) thường yêu cầu bộ nhớ lớn để lưu trữ các bản đồ đặc trưng (feature maps) độ phân giải cao, do đó ít được ưu tiên trên các hệ thống FPGA tài nguyên thấp nếu không có bộ tăng tốc phần cứng chuyên dụng.

2.5.2. TRAINING VÀ INFERENCE

Để triển khai AI trên phần cứng, cần phân biệt rõ hai giai đoạn:

- **Huấn luyện (Training):** Quá trình tìm ra bộ trọng số (weights) tối ưu cho mô hình thông qua việc học từ dữ liệu lớn. Quá trình này yêu cầu tính toán dấu phẩy động (Float32/Float16) và thường thực hiện trên Server hoặc GPU mạnh.
- **Suy luận (Inference):** Quá trình sử dụng mô hình đã huấn luyện để đưa ra dự đoán trên dữ liệu mới. Tại thiết bị biên (Edge), suy luận yêu cầu độ trễ thấp và thường sử dụng kỹ thuật lượng tử hóa để chạy trên phần cứng số nguyên (Integer hardware) như RISC-V soft-core.

2.5.3. PHÂN LOẠI AI VÀ MỐI LIÊN HỆ VỚI DEEP LEARNING

Trí tuệ nhân tạo là một thuật ngữ rộng, bao hàm nhiều cấp độ:

- **AI hẹp (Narrow AI):** Các hệ thống được thiết kế cho một tác vụ cụ thể (như nhận dạng ảnh trong đề tài này).
- **Học máy (Machine Learning - ML):** Tập con của AI, nơi máy tính học từ dữ liệu thay vì được lập trình quy tắc cứng.
- **Học sâu (Deep Learning - DL):** Tập con của ML, sử dụng các mạng nơ-ron nhiều lớp để trích xuất đặc trưng tự động.

Chi tiết về cấu trúc và nguyên lý hoạt động của Mạng nơ-ron nhân tạo (ANN) và Học sâu sẽ được trình bày chuyên sâu tại Mục 2.6.

2.5.4. EDGE AI

KHÁI NIỆM

Edge AI là sự kết hợp giữa thuật toán AI và tính toán biên (Edge Computing), cho phép xử lý dữ liệu cục bộ ngay trên thiết bị phần cứng mà không cần gửi dữ liệu về trung tâm dữ liệu (Cloud).

ƯU ĐIỂM SO VỚI CLOUD AI

- **Độ trễ thấp (Low Latency):** Phản hồi tức thời, phù hợp cho các ứng dụng thời gian thực (ví dụ: xe tự lái, robot).
- **Bảo mật và Quyền riêng tư:** Dữ liệu hình ảnh nhạy cảm không rời khỏi thiết bị, giảm nguy cơ bị tấn công đường truyền.
- **Tiết kiệm băng thông:** Không cần truyền tải luồng video liên tục lên máy chủ, giảm tải cho hạ tầng mạng.
- **Độ tin cậy:** Hệ thống vẫn hoạt động bình thường ngay cả khi mất kết nối Internet.

CÁC FRAMEWORK PHỔ BIẾN CHO EDGE AI

- **TensorFlow Lite (TFLite):** Phiên bản rút gọn của TensorFlow, tối ưu cho thiết bị di động và nhúng. TFLite hỗ trợ chuyển đổi mô hình sang định dạng FlatBuffer nhẹ và cung cấp các toán tử tối ưu cho vi xử lý ARM và RISC-V.
- **Vitis AI:** Nền tảng phát triển của AMD-Xilinx, chuyên dụng để tối ưu hóa mô hình AI chạy trên các kiến trúc DPU (Deep Learning Processor Unit) của FPGA Xilinx.

- **ONNX Runtime:** ONNX (Open Neural Network Exchange) là định dạng trung gian giúp chuyển đổi mô hình giữa các framework (PyTorch sang TensorFlow). ONNX Runtime cho phép chạy suy luận hiệu năng cao trên nhiều nền tảng phần cứng khác nhau.

2.6. MẠNG NƠ-RON NHÂN TẠO (ARTIFICIAL NEURAL NETWORKS)

Mạng nơ-ron nhân tạo (ANN) là nền tảng của Deep Learning, lấy cảm hứng từ cơ chế truyền dẫn tín hiệu điện hóa giữa các nơ-ron trong bộ não sinh học. Trước khi đi sâu vào kiến trúc CNN được sử dụng trong đề tài, cần xem xét tổng quan các kiến trúc mạng nơ-ron phổ biến để thấy rõ sự phù hợp của từng loại đối với các tác vụ cụ thể.[4]

2.6.1. CÁC KIẾN TRÚC MẠNG NƠ-RON PHỔ BIẾN

MẠNG NƠ-RON TRUYỀN THẲNG (FEEDFORWARD NEURAL NETWORK - MLP)

Đây là dạng đơn giản nhất của ANN, còn được gọi là Multi-Layer Perceptron (MLP). Trong kiến trúc này, thông tin di chuyển theo một chiều từ lớp đầu vào (Input), qua các lớp ẩn (Hidden layers) đến lớp đầu ra (Output) mà không có chu trình quay lại.

- **Đặc điểm:** Các nơ-ron ở lớp trước kết nối đầy đủ (fully connected) với nơ-ron lớp sau.
- **Hạn chế với hình ảnh:** Khi xử lý ảnh, MLP làm mất thông tin không gian (spatial structure) do phải duỗi phẳng ảnh thành vector 1 chiều. Ngoài ra, số lượng tham số bùng nổ quá nhanh dẫn đến quá tải bộ nhớ và hiện tượng quá khớp (overfitting).

CÁC KIẾN TRÚC KHÁC

Ngoài MLP, lĩnh vực mạng nơ-ron còn có Mạng nơ-ron Hồi quy (RNN, với các biến thể LSTM/GRU) chuyên cho dữ liệu chuỗi thời gian, và Vision Transformers (ViT) áp dụng cơ chế Self-Attention. Tuy nhiên, RNN không hiệu quả trong việc trích xuất đặc trưng không gian 2D và khó song song hóa, còn ViT có độ phức tạp $O(N^2)$ tiêu tốn tài nguyên rất lớn – cả hai đều không phù hợp với ràng buộc của một soft-core RISC-V trên FPGA tài nguyên hạn chế. Do đó, đồ án lựa chọn Mạng nơ-ron Tích chập (CNN) làm kiến trúc chính, sẽ được trình bày tại mục 2.6.2.

2.6.2. MẠNG NƠ-RON TÍCH CHẬP (CNN)

Dựa trên phân tích trên, CNN là kiến trúc cân bằng nhất giữa hiệu năng và chi phí tính toán cho các thiết bị biên. CNN là kiến trúc mạng chuyên

biệt cho dữ liệu dạng lưới (grid-like topology) như hình ảnh. Một mạng CNN điển hình bao gồm ba loại lớp chính:

LỚP TÍCH CHẬP (CONVOLUTIONAL LAYER)

Đây là lớp quan trọng nhất, thực hiện trích xuất đặc trưng cục bộ và bảo toàn tính chất không gian của ảnh. Phép tính tích chập là việc trượt một cửa sổ nhỏ (Kernel/Filter) trên toàn bộ ảnh đầu vào và thực hiện phép nhân chập. Về mặt toán học, giá trị đầu ra Y tại vị trí (i, j) được tính như sau:

$$Y[i, j] = \sum_m \sum_n X[i + m, j + n] \cdot K[m, n] + b \tag{2.1}$$

Trong đó: X là ma trận đầu vào, K là ma trận Kernel, và b là bias. Trên các vi xử lý RISC-V hỗ trợ DSP (như Pulpino), phép toán này có thể được tăng tốc bằng các lệnh SIMD (Single Instruction Multiple Data), cho phép thực hiện 4 phép nhân 8-bit trong một chu kỳ xung nhịp.

HÀM KÍCH HOẠT (ACTIVATION FUNCTION)

Hàm kích hoạt giúp đưa tính phi tuyến vào mô hình, cho phép mạng học được các đặc trưng phức tạp. Hàm phổ biến nhất trong CNN là **ReLU (Rectified Linear Unit)**:

$$f(x) = \max(0, x) \tag{2.2}$$

ReLU có ưu điểm lớn về mặt tính toán trên phần cứng nhúng vì chỉ cần một lệnh so sánh đơn giản (nếu $x < 0$ thì gán bằng 0), không tốn kém tài nguyên như hàm Sigmoid hay Tanh (vốn yêu cầu tính toán số mũ thực rất nặng nề cho soft-core).

LỚP GỘP (POOLING LAYER)

Dùng để giảm kích thước không gian của bản đồ đặc trưng (down-sampling), giúp giảm số lượng tham số và tính toán, đồng thời giúp mô hình bất biến với các dịch chuyển nhỏ (Translation Invariance) của đối tượng. Hai kỹ thuật phổ biến là Max Pooling (lấy giá trị lớn nhất) và Average Pooling (lấy giá trị trung bình).

2.6.3. KỸ THUẬT LƯỢNG TỬ HÓA (QUANTIZATION)

Lượng tử hóa là kỹ thuật cốt lõi để triển khai AI trên các vi xử lý RISC-V soft-core không có bộ xử lý dấu phẩy động (FPU) mạnh mẽ.

KHÁI NIỆM

Kỹ thuật này chuyển đổi các trọng số (weights) và giá trị kích hoạt (activations) từ dạng số thực 32-bit (Float32) sang số nguyên 8-bit (Int8). Công thức chuyển đổi tổng quát:

$$r = S \times (q - Z) \tag{2.3}$$

Trong đó: r là giá trị thực, q là giá trị lượng tử hóa (int8), S là tỉ lệ (Scale factor), và Z là điểm không (Zero-point).

LỢI ÍCH TRÊN FPGA/RISC-V

- **Giảm bộ nhớ:** Kích thước mô hình giảm 4 lần (từ 32-bit xuống 8-bit), giúp mô hình vừa vặn với bộ nhớ on-chip (BRAM/TCM) hạn hẹp của FPGA, giảm độ trễ truy cập bộ nhớ ngoài (DDR).

- **Tăng tốc độ:** Các phép tính số học trên số nguyên (Integer Arithmetic) nhanh hơn và tiêu thụ ít năng lượng hơn nhiều so với số thực. Các nhân RISC-V như RI5CY hỗ trợ thực hiện 4 phép nhân cộng Int8 trong một chu kỳ máy.

2.6.4. CÁC KỸ THUẬT TỐI ƯU HÓA KHÁC

KỸ THUẬT CẮT TỈA

Pruning loại bỏ các kết nối (trọng số) có giá trị gần bằng 0 hoặc ít ảnh hưởng đến kết quả đầu ra. Kỹ thuật này biến ma trận trọng số dày đặc (Dense) thành ma trận thưa (Sparse), giúp giảm đáng kể khối lượng tính toán và bộ nhớ lưu trữ mà không làm giảm nhiều độ chính xác.

KỸ THUẬT IM2COL (IMAGE-TO-COLUMN)

Để tối ưu hóa phép tính tích chập trên phần cứng, kỹ thuật *im2col* thường được sử dụng. Nó biến đổi ma trận ảnh đầu vào thành các cột, chuyển bài toán tích chập phức tạp (Convolution) thành bài toán nhân ma trận (GEMM - General Matrix Multiply). GEMM là phép toán đã được tối ưu hóa rất tốt trên các thư viện tính toán (như BLAS) và tận dụng triệt để khả năng xử lý song song.

Trong đồ án này, bộ tăng tốc Conv-CNN v2.0 không sử dụng *im2col* vì áp dụng kiến trúc *streaming dataflow* với line buffer + window register: dữ liệu được trượt qua window 3×3 và đưa vào trực tiếp 9 PE MAC, tránh chi phí biến đổi và sao chép bộ nhớ của *im2col*.

3

CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN

3.1. PULPISSIMO

3.1.1. TỔNG QUAN

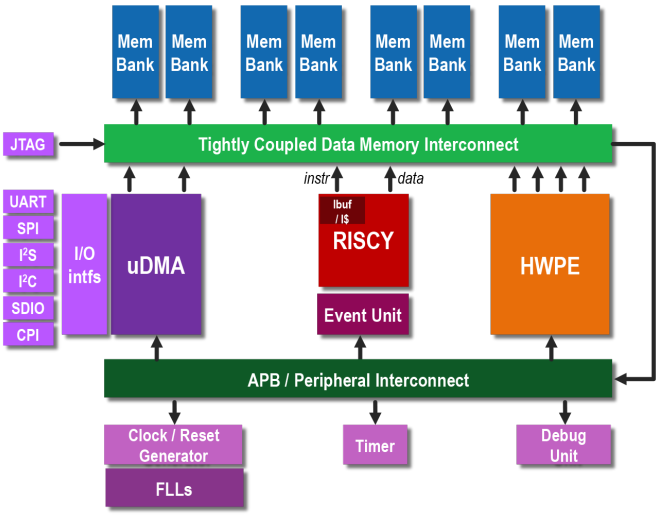
Pulpissimo (PULP Independent Single-stream Shared-memory Integrated Subsystem for MOtion control) là một hệ thống vi điều khiển (Microcontroller System) tiên tiến dựa trên kiến trúc RISC-V 32-bit, được phát triển bởi nhóm nghiên cứu PULP (Parallel Ultra Low Power) thuộc ETH Zürich và Đại học Bologna.

Mục tiêu thiết kế của Pulpissimo là cung cấp một nền tảng phần cứng linh hoạt, tiết kiệm năng lượng nhưng vẫn đảm bảo hiệu năng tính toán cao cho các ứng dụng IoT và Edge AI yêu cầu băng thông lớn. Khác với các vi điều khiển truyền thống, Pulpissimo không chỉ là một lõi CPU đơn lẻ mà là một hệ thống SoC (System-on-Chip) hoàn chỉnh với kiến trúc bộ nhớ và ngoại vi thông minh [5].

3.1.2. CÁC ĐẶC ĐIỂM KỸ THUẬT NỔI BẬT

PULPissimo bao gồm các thành phần chính sau:

- Bộ xử lý chính: Hoặc bộ xử lý RI5CY hoặc bộ xử lý Ibex
- Hệ thống con uDMA: Hệ thống con đầu vào/đầu ra tự động để quản lý thiết bị ngoại vi hiệu quả
- Hệ thống con bộ nhớ: Bao gồm ROM khởi động và bộ nhớ L2



Hình 3.1: Kiến trúc Pulpissimo

- Hệ thống sự kiện và ngắt: Bộ điều khiển ngắt đơn giản để quản lý các sự kiện hệ thống
- Thiết bị ngoại vi: Bao gồm SPI, I2S, giao diện camera, I2C, UART, Hyperbus và GPIO
- Bộ xử lý phần cứng (HWPE): Hỗ trợ các bộ tăng tốc phần cứng tùy chỉnh

LÕI XỬ LÝ CV32E40P (RI5CY)

Trung tâm của Pulpissimo là nhân xử lý RI5CY (hiện nay được chuẩn hóa thành CV32E40P bởi OpenHW Group). Đây là một hiện thực hóa (implementation) của RISC-V với đường ống 4 tầng, được tối ưu hóa đặc biệt cho xử lý tín hiệu số (DSP). Điểm đặc biệt của nhân này là việc hỗ trợ các mở rộng tập lệnh không tiêu chuẩn (Non-standard Extensions) giúp tăng tốc đáng kể các thuật toán AI:

- **Hardware Loops:** Cho phép thực hiện các vòng lặp mà không tốn chu kỳ máy cho việc kiểm tra điều kiện và rẽ nhánh.
- **SIMD (Single Instruction Multiple Data):** Hỗ trợ tính toán song song trên các vector nhỏ (ví dụ: thực hiện 2 phép cộng 16-bit hoặc 4 phép cộng 8-bit trong 1 chu kỳ). Đây là tính năng then chốt để xử lý dữ liệu lượng tử hóa (Quantized Data) trong mạng nơ-ron.

- **Bit Manipulation:** Các lệnh thao tác bit giúp tối ưu hóa việc đóng gói và giải nén dữ liệu trọng số.

HỆ THỐNG BỘ NHỚ VÀ UDMA

Để giải quyết bài toán "nghẽn cổ chai"dữ liệu, Pulpissimo sử dụng bộ điều khiển truy cập bộ nhớ trực tiếp thông minh (micro-DMA hay uDMA). uDMA cho phép truyền dữ liệu trực tiếp từ các ngoại vi tốc độ cao (như Camera Interface, SPI) vào bộ nhớ nội (L2 Memory) mà không cần sự can thiệp của CPU. Cơ chế này giúp CPU có thể chuyển sang trạng thái ngủ (Sleep Mode) trong khi dữ liệu đang được thu thập, giúp tiết kiệm năng lượng tối đa .

3.2. MÔ HÌNH HIỆU QUẢ CHO THIẾT BỊ BIÊN

Bên cạnh các vi xử lý mở, hướng nghiên cứu thứ hai để đưa AI lên thiết bị biên là tối ưu hóa kiến trúc mạng nơ-ron thay vì phần cứng. Hai trường phái tiêu biểu được tóm tắt dưới đây để làm rõ vị trí thiết kế của đồ án.

Mạng nơ-ron Nhị phân (BNN). Courbariaux và cộng sự [6] đã chứng minh khả năng huấn luyện các mạng nơ-ron sâu với trọng số và kích hoạt ép buộc về $\{+1, -1\}$ (1-bit). Lợi thế lớn nhất của BNN trên FPGA là phép tích chập được thay bằng phép logic XNOR + Popcount, có thể thực hiện trực tiếp bằng LUT mà không cần DSP. Tuy nhiên, BNN gặp giảm độ chính xác đáng kể trên các tập dữ liệu phức tạp (ImageNet). Đồ án tham khảo nguyên lý tối ưu phần cứng của BNN nhưng lựa chọn lượng tử hóa INT8 chuẩn TFLite [7] để đạt cân bằng tốt hơn giữa độ chính xác và hiệu năng.

MobileNet và Depthwise Separable Convolution. Howard và cộng sự [8] đề xuất kiến trúc MobileNet sử dụng *depthwise separable convolution* (tách Conv $K \times K$ thành Depthwise Conv + Pointwise 1×1) để giảm khoảng 8–9 lần số phép MAC so với Conv chuẩn cùng kích thước. MobileNet trở thành kiến trúc chuẩn cho thị giác máy trên thiết bị di động. Tuy nhiên, depthwise convolution có *dataflow ngược* so với Conv chuẩn (mỗi cin → 1 output thay vì sum across cin), đòi hỏi MUX adder tree riêng và chế độ FSM mới ở bộ tăng tốc. Trong phạm vi đồ án, đề tài chọn kiến trúc `vgg-tiny` đơn giản hơn (chỉ Conv 3×3 chuẩn) để tập trung vào việc kiểm chứng đồng thiết kế phần cứng-phần mềm; hỗ trợ depthwise được dành cho hướng phát triển tiếp theo.

3.3. GIA TỐC PHẦN CỨNG TRÊN TẬP LỆNH MỞ RỘNG RISC-V (ISA EXTENSIONS)

Một trong những hướng tiếp cận phổ biến để gia tốc mạng nơ-ron trên các thiết bị biên (Edge Devices) là can thiệp trực tiếp vào kiến trúc tập lệnh (ISA - Instruction Set Architecture) của vi xử lý. Công trình nghiên cứu *"FPGA-Accelerated RISC-V ISA Extensions for Efficient Neural Network Inference on Edge Devices"* là một minh chứng tiêu biểu cho trường phái thiết kế liên kết chặt chẽ (Tightly-Coupled Architecture). [9]

Nguyên lý hoạt động và Kiến trúc: Thay vì thiết kế một khối phần cứng gia tốc độc lập, phương pháp này tiến hành sửa đổi trực tiếp đường ống thực thi (Pipeline) và khối tính toán logic (ALU) bên trong lõi vi xử lý

RISC-V. Các tác giả định nghĩa thêm các chỉ thị máy (Assembly Instructions) hoàn toàn mới, chuyên biệt cho phép toán của mạng nơ-ron, điển hình là các lệnh nhân cộng tích lũy (MAC - Multiply-Accumulate) trên định dạng số nguyên lượng tử hóa (INT8). Khi bộ giải mã lệnh (Instruction Decoder) của CPU gặp các chỉ thị tùy biến này, nó sẽ điều hướng dữ liệu từ tập thanh ghi (Register File) dùng chung sang mạch phần cứng gia tốc vừa được cấy ghép.

Ưu điểm của phương pháp:

- **Độ trễ giao tiếp cực thấp (Zero-latency Communication):** Do khối gia tốc nằm ngay bên trong lõi CPU và sử dụng trực tiếp tài nguyên thanh ghi nội bộ, hệ thống loại bỏ hoàn toàn độ trễ phát sinh từ việc truyền tải dữ liệu qua các bus giao tiếp ngoại vi (như AXI hay APB).
- **Tối ưu hóa năng lượng (Energy Efficiency):** Việc giảm thiểu các thao tác đọc/ghi bộ nhớ ngoài giúp tiết kiệm đáng kể năng lượng, đáp ứng tốt các ràng buộc khắt khe của thiết bị Edge AI.

Thách thức và Giới hạn: Mặc dù mang lại hiệu năng tính toán tại chỗ cao, kiến trúc ISA Extensions đi kèm với sự đánh đổi lớn về độ phức tạp trong khâu phát triển:

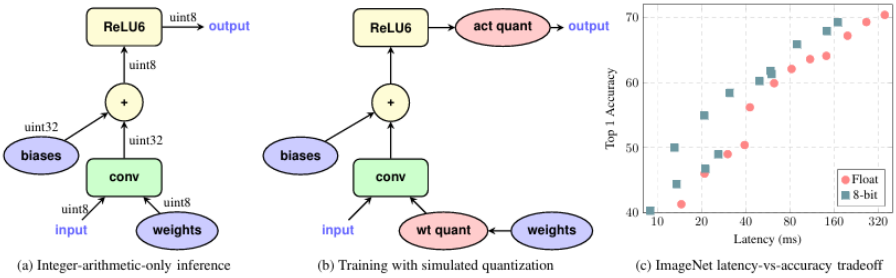
- **Xâm lấn lõi phần cứng (Hardware Intrusion):** Đòi hỏi nhà thiết kế phải can thiệp sâu vào mã nguồn (RTL) của bộ vi xử lý chuẩn, làm tăng rủi ro về mặt định tuyến (Routing) và có thể làm giảm tần số hoạt động tối đa (Fmax) của toàn hệ thống.
- **Giới hạn kích thước ma trận:** Khối gia tốc bị giới hạn không gian vật lý bên trong CPU và phụ thuộc vào băng thông của tập thanh ghi (thường chỉ 32-bit), khiến nó khó mở rộng để xử lý các lớp tích chập (Convolutional Layers) có kích thước lớn.
- **Chuỗi công cụ phần mềm (Toolchain Dependency):** Việc bổ sung lệnh mới bắt buộc phải tùy biến lại các trình biên dịch mã nguồn mở (như GCC hoặc LLVM) để phần mềm cấp cao (C/C++) có thể hiểu và biên dịch đúng các chỉ thị gia tốc này.

Kết luận đối chiếu: Việc phân tích phương pháp mở rộng tập lệnh (ISA Extensions) cho thấy rõ sự đánh đổi giữa độ trễ giao tiếp và tính linh hoạt của hệ thống. Đối với các mạng nơ-ron có quy mô lớn yêu cầu thông lượng bộ nhớ cao, một kiến trúc liên kết lỏng (Loosely-Coupled) dựa trên chuẩn giao tiếp bộ nhớ ánh xạ (Memory-Mapped) độc lập thường mang lại khả năng mở rộng tốt hơn mà không phá vỡ tính nguyên bản của lõi vi xử lý.

3.4. LƯỢNG TỬ HÓA VÀ HUẤN LUYỆN MẠNG NƠ-RON

Trong các hệ thống học sâu truyền thống, việc huấn luyện và suy luận thường được thực hiện trên định dạng số thực dấu phẩy động 32-bit (Float32) để đảm bảo độ chính xác cao nhất. Tuy nhiên, trên các thiết bị biên (Edge Devices) với tài nguyên hạn chế như FPGA hay vi xử lý nhúng RISC-V, các phép toán số thực tiêu tốn quá nhiều tài nguyên phần cứng và năng lượng.

Để giải quyết vấn đề này, đồ án áp dụng phương pháp lượng tử hóa số nguyên được đề xuất bởi *Jacob et al.* (Google) trong công trình *"Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference"* [7].



Hình 3.2: Quy trình lượng tử hóa và huấn luyện mạng nơ-ron theo Jacob et al.

3.4.1. CƠ CHẾ LƯỢNG TỬ HÓA AFFINE (AFFINE QUANTIZATION SCHEME)

Khác với các phương pháp lượng tử hóa logarit hay nhị phân, phương pháp của Jacob et al. sử dụng ánh xạ tuyến tính (Affine mapping) để chuyển đổi giữa giá trị thực (r) và giá trị nguyên 8-bit (q). Công thức ánh xạ được định nghĩa như sau:

$$r = S \times (q - Z) \tag{3.1}$$

Trong đó:

- r (Real value): Giá trị thực cần biểu diễn.
- q (Quantized value): Giá trị nguyên 8-bit ($q \in [0, 255]$ hoặc $[-128, 127]$).
- S (Scale factor): Hằng số tỉ lệ (dạng Float32 dương), xác định bước nhảy của lượng tử hóa.
- Z (Zero-point): Giá trị nguyên đóng vai trò là điểm "0"của số thực. Điều này cho phép biểu diễn chính xác giá trị 0 thực tế (thường xuất

hiện trong Padding hoặc ReLU) bằng một số nguyên, giúp duy trì độ chính xác của mạng.

Từ công thức trên, quá trình nhân chập (Convolution) giữa đầu vào x và trọng số w có thể được viết lại hoàn toàn dưới dạng số nguyên:

$$y = \sum(x \times w) \approx S_y \left(\sum(q_x - Z_x)(q_w - Z_w) + Z_y\right) \tag{3.2}$$

3.4.2. SUY LUẬN CHỈ DÙNG SỐ HỌC NGUYÊN (INTEGER-ARITHMETIC-ONLY INFERENCE)

Điểm đột phá trong nghiên cứu của Jacob et al. là kỹ thuật xử lý tham số S (Scale). Mặc dù S là số thực, nhưng trong quá trình suy luận, tích của các Scale ($M = S_x S_w / S_y$) được xấp xỉ bằng một phép nhân số nguyên và phép dịch bit (Bit-shift):

$$M \approx 2^{-n} \times M_0 \quad \text{(với } M_0 \text{ là số nguyên cố định)} \tag{3.3}$$

Kỹ thuật này cho phép loại bỏ hoàn toàn khối xử lý số thực (FPU) khỏi phần cứng. Mọi phép tính từ tích chập, cộng bias đến hàm kích hoạt đều được thực hiện bằng các đơn vị số học nguyên (ALU Integer) và các bộ dịch bit.

4

THIẾT KẾ KIẾN TRÚC HỆ THỐNG

4.1. TỔNG QUAN KIẾN TRÚC PLATFORM

Để đảm bảo tính linh hoạt, khả năng mở rộng và tái sử dụng, hệ thống được thiết kế dựa trên mô hình phân lớp chức năng. Kiến trúc này cho phép tách biệt giữa hạ tầng phần cứng vật lý và logic thực thi của ứng dụng AI.

chapter04/img/platform.png

Hình 4.1: Stack kiến trúc hệ thống Edge-AI

- Lớp 1: Hardware Layer (Cơ sở hạ tầng)Phân tích lý do chọn Zynq làm Host (Quản lý I/O tốt, chạy Linux) và CV32E40P làm Accelerator (Lõi nhỏ gọn, hỗ trợ tập lệnh IMC, tối ưu diện tích chip) .Nhấn mạnh vào khối AXI Interconnect và BRAM Controller (Phiên dịch viên ngôn ngữ bus) - Đây chính là xương sống của Platform.
- Lớp 2: Software / Runtime Layer (Hệ điều hành & Trình điều khiển) Trình bày luồng hoạt động (Data Flow): ARM đóng vai trò Master, dùng Memory Mapped I/O để đẩy dữ liệu. RISC-V đóng vai trò Slave, luôn ở trạng thái chờ lệnh (Polling) ARM chụp ảnh → Đẩy vào BRAM → Kích hoạt cờ (Flag) → RISC-V đọc cờ → Tính toán → Ghi kết quả → ARM đọc kết quả.
- Lớp 3: Application Layer (Tầng ứng dụng): Một nền tảng "mở". Hiện tại đang chạy nhận diện người (CNN), nhưng vì là Platform, ngày mai người dùng hoàn toàn có thể nạp firmware khác để chạy nhận diện âm thanh mà không cần sửa lại phần cứng.

4.1.1. LAYER 1: HARDWARE LAYER

Đây là tầng thấp nhất của nền tảng, thực thi các tác vụ tính toán cường độ cao và quản lý luồng dữ liệu thô. Thành phần chính bao gồm:

Hệ thống xử lý (Processing System - PS): Tận dụng cụm lõi ARM Cortex-A53 của Zynq UltraScale+ để điều phối toàn hệ thống.

Bộ gia tốc lõi mềm (Soft-core Accelerator): Lõi vi xử lý RISC-V CV32E40P kết hợp cùng khối tăng tốc phần cứng (CNN Accelerator) tùy chỉnh trên vùng Programmable Logic (PL).

Hệ thống di chuyển dữ liệu (Data Mover): Tích hợp khối DMA Controller và mạng lưới AXI4 Interconnect để vận chuyển dữ liệu giữa DDR4 và Shared BRAM với độ trễ thấp nhất, giảm thiểu gánh nặng cho CPU trung tâm.

4.1.2. LAYER 2: SOFTWARE / RUNTIME LAYER

Đóng vai trò là "cầu nối"(Middleware) giúp lớp ứng dụng có thể tương tác với phần cứng một cách minh bạch:

Host Environment: Hệ điều hành Linux chạy trên lõi ARM, cung cấp các Driver giao tiếp Camera, bộ nhớ và thư viện tiền xử lý ảnh (OpenCV).

Communication Protocol: Hiện thực hóa cơ chế Memory-Mapped I/O và giao thức bắt tay thông qua các cờ hiệu (Flags) trong bộ nhớ dùng chung.

Firmware Subsystem: Chứa mã nguồn Bare-metal tối ưu hóa cho RISC-V, bao gồm các thư viện toán tử AI cơ bản và trình điều khiển khối tăng

tốc phần cứng.

4.1.3. LAYER 3: APPLICATION LAYER

Đây là tầng cao nhất, quy định kịch bản sử dụng thực tế của hệ thống.

- **Ứng dụng kiểm chứng (Use-case):** Hiện tại, tầng này đang thực thi cấu trúc mạng **MobileNetV2** (lượng tử hóa INT8) để giải quyết bài toán phát hiện người (Human Detection) trên tập dữ liệu INRIA.
- **Tính linh hoạt trong cùng miền ứng dụng (Domain-Specific Flexibility):** Nhờ thiết kế phân lớp, người dùng có thể dễ dàng chuyển đổi hệ thống sang giải quyết bài toán khác trong cùng lĩnh vực thị giác máy tính (ví dụ: chuyển từ MobileNet sang nhận diện biển báo giao thông bằng kiến trúc CNN khác). Việc này chỉ yêu cầu cập nhật tệp trọng số và mã nguồn Firmware (C/C++) ở Layer 2, mà hoàn toàn không cần can thiệp hay tổng hợp lại kiến trúc vật lý ở Layer 1.

4.2. KIẾN TRÚC DỊ THỂ

Hệ thống Edge-AI được thiết kế dựa trên kiến trúc không đồng nhất (Heterogeneous Architecture) của nền tảng Zynq UltraScale+ MPSoC trên bo mạch Kria KV260. Thiết kế tận dụng mô hình đồng thiết kế phần cứng/phần mềm (Hardware/Software Co-design) để tối ưu hóa hiệu năng xử lý.

Mô hình tổng thể được chia thành hai miền xử lý chính:

- Processing System
- Khối Cầu nối
- Programmable Logic

Như thể hiện tại Hình 4.2, hệ thống được phân chia thành ba miền xử lý chính với các vai trò chuyên biệt:

- **Phân hệ Xử lý (Processing System - PS):** Đóng vai trò là khối điều khiển trung tâm (Host), hoạt động trên nền tảng vi xử lý ARM Cortex-A53 lõi tứ. PS chịu trách nhiệm:
  - Quản lý tài nguyên hệ thống và khởi chạy quy trình boot.
  - Giao tiếp với các ngoại vi tiêu chuẩn như Camera (qua giao diện MIPI CSI-2) và Màn hình.
  - Thực hiện các bước tiền xử lý hình ảnh (Preprocessing) và quản lý bộ nhớ đệm dữ liệu (Data Buffering) trên DDR4.

chapter04/img/arch.png

Hình 4.2: Kiến trúc hệ thống tổng quan

- **Khối Giao tiếp Liên kết (Interconnect Fabric):** Đóng vai trò là cầu nối dữ liệu tốc độ cao, sử dụng giao thức chuẩn công nghiệp AMBA AXI4. Khối này bao gồm các bộ chuyển mạch thông minh (AXI SmartConnect) để:
  - Phân tách luồng điều khiển (Control Plane) băng thông thấp và luồng dữ liệu (Data Plane) băng thông cao.
  - Đảm bảo đồng bộ hóa giữa hai miền xung nhịp khác nhau của PS và PL.
  - Cho phép truy cập bộ nhớ chia sẻ (Shared Memory) một cách trong suốt giữa các vi xử lý.

- **Phân hệ Logic Khả trình (Programmable Logic - PL):** Đóng vai trò là bộ tăng tốc tính toán (Accelerator). Tại đây, tài nguyên FPGA được cấu hình để triển khai một Hệ thống trên Chip (SoC) con, bao gồm:
  - **Lõi vi xử lý mềm RISC-V (PULP Subsystem):** Điều phối luồng thực thi thuật toán AI.
  - **Bộ tăng tốc phần cứng (CNN Accelerator):** Thực hiện các phép toán tích chập chuyên sâu.
  - **Cơ chế truy cập bộ nhớ trực tiếp (uDMA):** Tối ưu hóa việc di chuyển dữ liệu ảnh mà không làm gián đoạn CPU.

Sự kết hợp chặt chẽ giữa ba thành phần này cho phép hệ thống hoạt động theo cơ chế đường ống (Pipelining), đảm bảo khả năng xử lý hình ảnh thời gian thực với độ trễ thấp nhất.

4.3. THIẾT KẾ PROCESSING SYSTEM

Đây là phần cứng có sẵn (Hard-core) trên chip Zynq UltraScale+ của board Kria KV260.

- ARM Cortex-A53: Đóng vai trò là CPU chủ (Host). Nó có thểchạy hệ điều hành (Linux hoặc Bare-metal) để quản lý toàn bộ hệ thống. Nhiệm vụ của nó là giao tiếp với Camera (qua cổng MIPI), nhận ảnh, tiền xử lý (resize, crop) và lưu trữ ảnh vào bộ nhớ lớn.

- DDR4 Controller (Shared Memory): Đây là kho chứa dữ liệu chung. Vì bộ nhớ trong của RISC-V rất nhỏ (chỉ vài chục KB), toàn bộ dữ liệu ảnh đầu vào (Image Buffer) phải được lưu ở đây (4GB RAM của Kria).

PS đóng vai trò "Master"điều phối, gửi lệnh "Bắt đầu"(Start) cho khối PL khi đã có dữ liệu ảnh sẵn sàng.

4.3.1. CẤU HÌNH CHI TIẾT CÁC PHÂN HỆ

KHỐI XỬ LÝ ỨNG DỤNG (APU CONFIGURATION)

Trung tâm của PS là cụm vi xử lý Quad-core ARM Cortex-A53 hoạt động ở tần số 1.2 GHz. Trong thiết kế này, APU được cấu hình để:

- **Chế độ hoạt động:** Chạy hệ điều hành PetaLinux (dựa trên Yocto Project) để tận dụng các driver có sẵn cho Camera MIPI và ngăn xếp mạng (Network Stack).

- **Quản lý Boot:** Sử dụng giao diện SD Card (SD1) kết nối với các chân MIO để nạp bitstream cho FPGA và khởi động hệ thống.

CẤU HÌNH GIAO DIỆN BỘ NHỚ (DDR CONFIGURATION)

Bộ điều khiển DDR4 (DDRC) được kích hoạt để quản lý 4GB RAM tích hợp trên Kria SOM. Cấu hình này rất quan trọng vì nó đóng vai trò là "Shared Memory"cho cả hệ thống:

- **Data Width:** 64-bit (hỗ trợ băng thông cao).

- **AXI Port QoS:** Cổng S_AXI_HP0 được ưu tiên mức Quality of Service (QoS) cao để đảm bảo uDMA không bị nghẽn khi ghi dữ liệu ảnh, tránh hiện tượng rớt khung hình (frame drop).

CẤU HÌNH GIAO DIỆN PS-PL (INTERFACE CONFIGURATION)

Để kết nối với miền Logic khả trình (nơi chứa RISC-V và Accelerator), hai cổng giao tiếp AXI quan trọng được kích hoạt:

1. **M_AXI_HPM0_FPD (Master):**
   - *Chức năng:* Cổng Master hiệu năng cao miền công suất thấp (Full Power Domain).
   - *Nhiệm vụ:* Dùng để CPU ARM gửi tín hiệu điều khiển, cấu hình thanh ghi và nạp firmware vào bộ nhớ lệnh của RISC-V.
   - *Độ rộng:* 32-bit (phù hợp với bus AXI-Lite).
2. **S_AXI_HP0_FPD (Slave):**
   - *Chức năng:* Cổng Slave hiệu năng cao (High Performance).
   - *Nhiệm vụ:* Cho phép các Master bên phía FPGA (cụ thể là uDMA) truy cập trực tiếp vào RAM DDR4.
   - *Độ rộng:* 128-bit (để tối đa hóa tốc độ truyền tải ảnh).

CẤU HÌNH XUNG NHỊP (CLOCK CONFIGURATION)

Khối PS cung cấp nguồn xung nhịp tham chiếu cho toàn bộ logic bên phía PL thông qua chân `pl_clk0`. Tần số được thiết lập là **100 MHz**, đảm bảo sự cân bằng giữa tốc độ xử lý của nhân RISC-V và mức tiêu thụ năng lượng.

4.4. THIẾT KẾ HỆ THỐNG GIAO TIẾP VÀ BỘ NHỚ (INTERCONNECT & MEMORY HIERARCHY)

Trong kiến trúc SoC dị thể, hiệu năng của hệ thống không chỉ phụ thuộc vào tốc độ xử lý của từng nhân (Core) mà còn phụ thuộc vào băng thông và độ trễ của hệ thống kết nối. Mục này trình bày thiết kế chi tiết mạng lưới giao tiếp giữa PS và PL, cùng với quy hoạch không gian bộ nhớ chung.

chapter04/img/axi_arch.png

Hình 4.3: Sơ đồ khối hệ thống giao tiếp AXI giữa PS và PL.

4.4.1. CƠ CHẾ AXI SMARTCONNECT (PS-PL BRIDGE)

Để kết nối miền xử lý ứng dụng (PS) hoạt động ở tần số cao và miền logic khả trình (PL) hoạt động ở tần số thấp, hệ thống sử dụng khối IP **AXI

SmartConnect**. Đây không chỉ là dây dẫn đơn thuần mà là một ma trận chuyển mạch thông minh (Switch Fabric) đảm nhiệm ba chức năng:

- **Clock Domain Crossing (CDC):** Đồng bộ hóa tín hiệu giữa miền xung nhịp PS (1.2 GHz) và miền PL (100 MHz) bằng các bộ đệm FIFO bất đồng bộ, đảm bảo không xảy ra lỗi Metastability.
- **Width Conversion:** Chuyển đổi độ rộng dữ liệu tự động giữa bus hệ thống 128-bit (phía DDR) và bus ngoại vi 32-bit (phía RISC-V).
- **Arbitration:** Phân giải tranh chấp khi nhiều Master cùng truy cập bộ nhớ.

Hệ thống được thiết kế với hai kênh giao tiếp vật lý tách biệt:

KÊNH ĐIỀU KHIỂN (CONTROL PATH - AXI4-LITE)

Đây là tuyến giao tiếp từ **Master (ARM Cortex-A53)** đến **Slave (PULP Subsystem)**.

- **Giao thức:** AXI4-Lite 32-bit. Đây là phiên bản rút gọn của AXI4, không hỗ trợ truyền tải dạng Burst.
- **Mục đích:** Tối ưu hóa cho các giao dịch đơn lẻ, độ trễ thấp.
- **Chức năng:** Dùng để CPU ARM thực hiện các tác vụ quản trị như:
  - Cấu hình thanh ghi điều khiển của bộ tăng tốc CNN (Trọng số, Bias, Kích thước ảnh).
  - Gửi tín hiệu kích hoạt (Trigger) quá trình suy luận.
  - Reset mềm (Soft-reset) khối PL trong trường hợp treo hệ thống.

KÊNH DỮ LIỆU (DATA PATH - AXI4-FULL)

Đây là tuyến giao tiếp từ **Master (uDMA Controller)** đến **Slave (DDR4 Controller)**.

- **Giao thức:** AXI4-Full 128-bit.
- **Mục đích:** Tối ưu hóa băng thông (Throughput).
- **Chức năng:** Hỗ trợ cơ chế *Burst Transaction*, cho phép uDMA đọc/ghi liên tiếp tới 256 gói dữ liệu chỉ với một địa chỉ bắt đầu. Điều này cực kỳ quan trọng để đảm bảo cung cấp đủ dữ liệu pixel cho mạng MobileNet hoạt động thời gian thực (Real-time).

4.4.2. QUY HOẠCH KHÔNG GIAN BỘ NHỚ (SYSTEM MEMORY MAP)

Hệ thống quy hoạch không gian bộ nhớ thống nhất giữa hai góc nhìn của lõi RISC-V (32-bit flat) và lõi ARM (PS), với nguyên tắc **cùng địa chỉ logic** cho cùng một thiết bị slave để đơn giản hóa việc trao đổi con trỏ giữa hai master. Bảng 4.1 tổng hợp bản đồ bộ nhớ chi tiết.

Bảng 4.1: Bản đồ bộ nhớ hệ thống (System Memory Map)

Địa chỉ	**Kích thước**	**Slave**	**Chức năng**
Góc nhìn RISC-V (qua `m_axi_instr` + `m_axi_data`)			
`0x0000_0000`	64 KB	I-BRAM Port A	Instruction fetch (`.tex` `.rodata`, vector reset).
`0xB000_0000`	64 KB	`axi_dma_0` S_AXI_LITE	Thanh ghi điều khi DMA (MM2S, S2MI CTRL/STATUS).
`0xB001_0000`	64 KB	`conv_cnn_0` S00_AXI	Thanh ghi điều khiển bộ tăr tốc CNN (GEOMETRY, CTR STATUS, CHANNELS, M_Q3 SHIFT_ZP).
`0xB004_0000`	256 KB	D-BRAM Port A	`.data`, `.bss`, stack và các thar ghi handshake chia sẻ với ARM
Góc nhìn ARM PS (qua `M_AXI_HPM0/HPM1_FPD`)			
`0xA000_0000`	64 KB	I-BRAM Port B	ARM nạp/reload `firmware.b` vào BRAM lệnh của RISC-V.
`0xB001_0000`	64 KB	`conv_cnn_0` S00_AXI	ARM cấu hình trực tiếp bộ tăr tốc nếu cần.
`0xB004_0000`	256 KB	D-BRAM Port B	Vùng shared memory ARM RISC-V (handshake regs, lay pointers, kết quả).
Góc nhìn DMA (qua `S_AXI_HPC0_FPD`, cache-coherent)			
`0x0000_0000`	2 GB	DDR_LOW	IFM/Weights/OFM buffe cấp phát bởi ARM qu `pynq.allocate` (CMA).

Đặc điểm thiết kế quan trọng:

- **I-BRAM ánh xạ kép** (`0x0000_0000` từ RISC-V, `0xA000_0000` từ ARM): cho phép ARM nạp lại firmware mà không cần halt RISC-V.
- **D-BRAM ánh xạ đồng địa chỉ** (`0xB004_0000` từ cả hai phía): tiện cho debug lock-step và trao đổi pointer trực tiếp.

- **Bộ nhớ DDR riêng cho DMA**: vùng DDR_LOW chỉ được DMA truy cập (đã *Exclude* BRAM khỏi DMA route trong Vivado Address Editor để tránh xung đột); ARM chỉ định địa chỉ vật lý qua `pynq.allocate` và truyền pointer xuống RISC-V qua D-BRAM.

BỐ CỤC THANH GHI HANDSHAKE TRONG D-BRAM PORT B

Thay vì sử dụng một thanh ghi mailbox đơn lẻ, đề tài thiết kế một bố cục thanh ghi nhiều trường tại đầu vùng D-BRAM Port B (offset từ `0xB004_0000`) để trao đổi đầy đủ thông tin điều khiển và kết quả. Bảng 4.2 mô tả layout các thanh ghi:

Bảng 4.2: Bố cục thanh ghi chia sẻ trong D-BRAM Port B

Offset	Tên	Hướng	Mô tả
`+0x00`	`CMD_FROM_ARM`	ARM → RISC-V	`0x01` = START, `0x00` = idle.
`+0x04`	`STATUS_TO_ARM`	RISC-V → ARM	`0x00` IDLE / `0x01` BUSY / `0x02` DONE.
`+0x08`	`DATASET_ID`	ARM → RISC-V	0 = INRIA, 1 = Cats vs Dogs.
`+0x0C`	`RESULT_CLASS`	RISC-V → ARM	Kết quả argmax (tùy chọn; mặc định ARM tự argmax).
`+0x10`	`RESULT_CONF`	RISC-V → ARM	Confidence Q1.7 (tùy chọn).
`+0x18`	`IFM_PHYS_ADDR`	ARM → RISC-V	Địa chỉ vật lý DDR của buffer A (IFM, ping-pong scratch).
`+0x1C`	`OFM_PHYS_ADDR`	ARM → RISC-V	Địa chỉ vật lý DDR của buffer B (ping-pong scratch / OFM cuối).
`+0x20`	`WEIGHT_BASE`	ARM → RISC-V	Địa chỉ vật lý DDR của blob weights; RISC-V cộng `LAYERS[i].weight_offset` để ra phys addr per-layer.

CƠ CHẾ ĐỒNG BỘ HÓA

Quy trình bắt tay giữa ARM và RISC-V cho mỗi lần inference được mô tả như sau:

1. **ARM (chuẩn bị):** Cấp phát các CMA buffer cho weights, IFM ping-pong A, IFM ping-pong B; ghi pixel ảnh đã lượng tử hóa vào buffer A; ghi 3 thanh ghi địa chỉ `IFM_PHYS_ADDR`, `OFM_PHYS_ADDR`, `WEIGHT_BASE`.

2. **ARM (kích hoạt):** Ghi `0x01` vào `CMD_FROM_ARM`.

3. **RISC-V (xử lý):** Polling `CMD_FROM_ARM`; khi thấy `0x01` thì duyệt mảng `LAYERS[]`, với mỗi layer cấu hình DMA (MM2S đọc IFM/weights, S2MM ghi OFM), cấu hình `conv_cnn_0/S00_AXI`, kích hoạt và đợi xong.

4. **RISC-V (báo hoàn thành):** Sau layer cuối, ghi `STATUS_TO_ARM = 0x02` (DONE).

5. **ARM (đọc kết quả):** Polling `STATUS_TO_ARM`; khi DONE, gọi `.invalidate()` trên buffer cuối, đọc OFM, thực hiện GAP + softmax + argmax bằng phần mềm để có nhãn cuối cùng. Reset `CMD_FROM_ARM` về `0x00` để chuẩn bị cho frame tiếp theo.

4.5. THIẾT KẾ PROGRAMMABLE LOGIC

Logic Khả trình (PL) được thiết kế như một Hệ thống trên Chip (SoC) thu nhỏ hoạt động độc lập, dựa trên kiến trúc mở PULPissimo. Mục tiêu của phân hệ này là tạo ra một môi trường tính toán hiệu năng cao, độ trễ thấp để thực thi các tác vụ điều khiển và tăng tốc AI, giảm tải cho vi xử lý chính ARM.

4.5.1. KIẾN TRÚC TỔNG THỂ PHÂN HỆ PULP (PULP SUBSYSTEM)

Hình 4.4 trình bày sơ đồ kiến trúc chi tiết của khối PL. Thiết kế bao gồm ba thành phần cốt lõi: Lõi xử lý RISC-V, Hệ thống bộ nhớ TCDM phân tán, và Bộ điều khiển uDMA, được kết nối với nhau thông qua một mạng lưới giao tiếp băng thông rộng (Logarithmic Interconnect).

chapter04/img/pulp_arch.png

Hình 4.4: Sơ đồ kiến trúc nội bộ của Phân hệ PULP trên FPGA.

4.5.2. CẤU HÌNH NHÂN VI XỬ LÝ (RISC-V CORE CONFIGURATION)

Trung tâm điều khiển của PL là lõi IP RISC-V (dựa trên nhân CV32E40P). Lõi này được tham số hóa trước khi tổng hợp (Synthesis) để đạt sự cân bằng tối ưu giữa tài nguyên FPGA (LUTs) và hiệu năng tính toán:

Bảng 4.3: Thông số cấu hình RISC-V Soft-core

Tham số	**Giá trị**	**Ý nghĩa thiết kế**
Architecture	RV32IMCXpulp	Hỗ trợ tập lệnh nhân (M), nén (C) và mở rộng DSP/SIMD (Xpulp) cho AI.
Pipeline Stages	4	Đủ sâu để đạt tần số 100MHz, đủ ngắn để giảm độ trễ rẽ nhánh.
FPU	Disabled	Không sử dụng bộ xử lý dấu phẩy động để tiết kiệm diện tích (MobileNet đã được lượng tử hóa sang INT8).
Instruction Cache	4 KB	Kiểu Direct Mapped, giúp CPU nạp lệnh nhanh mà không cần truy cập RAM ngoài liên tục.
Boot Address	`0x1A00_0000`	Vị trí vector ngắt khởi động, trỏ tới vùng nhớ ROM khởi tạo.

4.5.3. HỆ THỐNG BỘ NHỚ TCDM VÀ INTERCONNECT

Điểm yếu chí tử của các hệ thống đa vi xử lý là hiện tượng tranh chấp bộ nhớ (Memory Contention). Thiết kế này giải quyết triệt để vấn đề trên bằng kiến trúc TCDM (Tightly Coupled Data Memory).

TỔ CHỨC BỘ NHỚ ĐAN XEN (INTERLEAVED MEMORY BANKING)

Thay vì sử dụng một khối Block RAM khổng lồ đơn nhất (64KB), bộ nhớ TCDM được chia nhỏ thành **16 ngân hàng (Banks)**, mỗi ngân hàng có dung lượng 4KB.

- **Nguyên lý:** Các địa chỉ liên tiếp nhau ($Addr, Addr + 4, Addr + 8...$) được lưu trữ lần lượt trên các Bank khác nhau ($Bank_0, Bank_1, Bank_2...$).
- **Lợi ích:** Cho phép truy cập song song. Ví dụ: RISC-V Core có thể đọc dữ liệu tại địa chỉ A (nằm ở Bank 0) cùng lúc uDMA ghi dữ liệu vào địa chỉ B (nằm ở Bank 1) mà không hề xảy ra xung đột.

LOGARITHMIC INTERCONNECT

Kết nối giữa các Master (Core, uDMA) và các Slave (Memory Banks) không phải là một Bus tuyến tính mà là một **Ma trận chuyển mạch (Switch Matrix)**.

- Mạng lưới này có độ trễ cực thấp (chỉ 1 chu kỳ xung nhịp).
- Đảm bảo băng thông cực đại: Tổng băng thông nội bộ có thể đạt tới $N \times 32$-bit/cycle (với N là số lượng Bank).

4.5.4. CƠ CHẾ TRUY CẬP BỘ NHỚ TRỰC TIẾP (UDMA)

Để giải phóng CPU khỏi tác vụ sao chép dữ liệu nhàm chán, một bộ điều khiển uDMA (Micro-DMA) chuyên dụng được tích hợp:

- **Chức năng:** uDMA đóng vai trò cầu nối giữa các ngoại vi tốc độ cao (như AXI4 port nối tới DDR4, SPI, Camera) và bộ nhớ TCDM.
- **Double Buffering:** uDMA hỗ trợ cấu hình hai vùng đệm (Buffer A và Buffer B). Khi đang xử lý dữ liệu ở Buffer A, uDMA sẽ tự động tải dữ liệu mới vào Buffer B trong nền (Background).
- **Kết nối với Accelerator:** uDMA cung cấp một luồng dữ liệu liên tục (Stream) cho khối tăng tốc CNN thông qua giao diện AXI-Stream, đảm bảo bộ tăng tốc luôn có dữ liệu để tính toán (tránh hiện tượng Starvation).

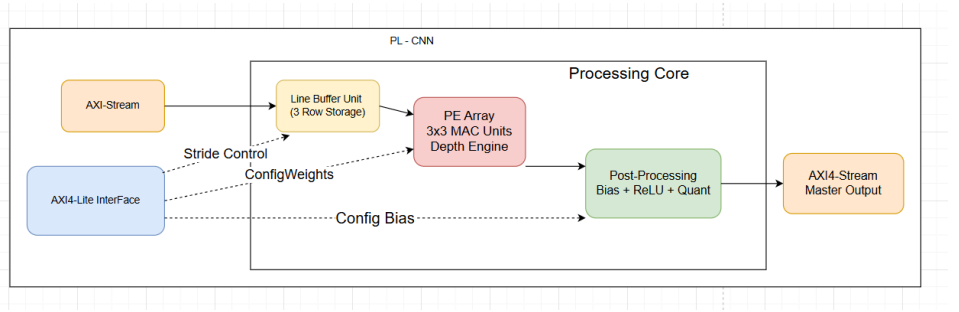
4.6. THIẾT KẾ KHỐI TĂNG TỐC PHẦN CỨNG (CNN ACCELERATOR)

Nằm trong PL, khối tăng tốc CNN được thiết kế để thực hiện các phép toán tích chập một cách song song và hiệu quả. Thiết kế này bao gồm các thành phần chính [10]

Đây là thành phần cốt lõi quyết định hiệu năng của toàn bộ hệ thống Edge-AI. Để đáp ứng yêu cầu tính toán thời gian thực của mạng MobileNetV1, nhóm thực hiện thiết kế một bộ tăng tốc phần cứng chuyên dụng (Domain-Specific Architecture - DSA) được tích hợp trong miền Logic khả trình (PL). Khối tăng tốc này được thiết kế để giảm tải (offload) các phép toán tích chập nặng nề nhất khỏi vi xử lý RISC-V, hoạt động theo cơ chế dòng dữ liệu (Dataflow) để tối đa hóa thông lượng.

4.6.1. KIẾN TRÚC TỔNG QUÁT KHỐI ACCELERATOR

Sơ đồ khối chức năng của bộ tăng tốc được mô tả trong Hình 4.5. Khối này giao tiếp với hệ thống thông qua hai giao diện chính:



Hình 4.5: Sơ đồ khối chức năng chi tiết của bộ tăng tốc CNN (Accelerator Core Architecture).

- **AXI4-Stream Slave:** Nhận dòng pixel ảnh đầu vào từ uDMA hoặc trực tiếp từ bộ nhớ đệm.
- **AXI4-Stream Master:** Đẩy dòng kết quả (Feature Maps) sau khi tính toán trở lại bộ nhớ.

4.6.2. CÁC THÀNH PHẦN CHỨC NĂNG CHI TIẾT

BỘ ĐỆM DÒNG (LINE BUFFER)

Để thực hiện tích chập 3×3, tại mỗi thời điểm cần truy cập đồng thời 3 hàng pixel liên tiếp. Nếu đọc trực tiếp từ DDR cho mỗi lần trượt cửa sổ

thì băng thông sẽ trở thành điểm nghẽn. Thiết kế sử dụng *line buffer* hai-bank dựa trên BRAM nội bộ với cơ chế *read-before-write parity-toggled*: khi một pixel mới được nạp vào bank này thì 2 hàng cũ vẫn được giữ ở bank kia. Cùng với khối `window_3x3` đứng sau (mảng thanh ghi $3 \times 3 \times$ `cin`), thiết kế tái sử dụng dữ liệu cục bộ và giảm số lần truy xuất bộ nhớ ngoài xuống còn 1 lần/pixel.

MẢNG TÍNH TOÁN PE VÀ ADDER TREE

Trung tâm tính toán của bộ tăng tốc gồm 9 PE (Processing Element) MAC (Multiply-Accumulate) hoạt động song song, tương ứng với 9 vị trí của kernel 3×3:

- Mỗi PE thực hiện phép nhân INT8×INT8 và cộng dồn vào thanh ghi tích lũy 32-bit. Cấu trúc pipelined với MREG + PREG cho phép Vivado ánh xạ trực tiếp vào khối DSP48E2.
- Đầu ra của 9 PE được tổng hợp qua *adder tree* 4-tầng để cho ra một giá trị tích chập 32-bit cho mỗi nhóm 9 pixel + 9 trọng số.
- Đối với layer có nhiều kênh đầu vào (`cin` > 1), bộ điều khiển sử dụng cơ chế *channel-interleaved per pixel*: tích lũy qua `cin` kênh trước khi xuất ra giá trị OFM cho mỗi tổ hợp $(y, x,$ `cnt_cout`$)$.

KHỐI LƯỢNG TỬ HÓA (REQUANTIZE)

Sau khi tổng chập, kết quả 32-bit cần được *re-quantize* về INT8 để khớp định dạng đầu vào của layer kế tiếp. Khối `requantize.sv` thực hiện đường ống 3-tầng theo công thức TFLite [7]:

$$y_q = \mathrm{sat}_{\mathrm{INT8}}\left(\left\lfloor \frac{(\sum x \cdot w + b) \cdot M_{q31} + 2^{30+n}}{2^{31+n}} \right\rfloor + z_{\mathrm{out}}\right) \tag{4.1}$$

trong đó M_{q31} là hệ số nhân Q31 (`int32`), n là số bit dịch phải ($n \in [0, 31]$), z_{out} là zero-point đầu ra. Cờ `has_relu` điều khiển việc kẹp giá trị âm về 0 (fused ReLU). Hệ số M_{q31} và n được tính sẵn trong pipeline huấn luyện và truyền xuống qua thanh ghi S00_AXI cho từng layer.

4.6.3. ĐẶC TÍNH THIẾT KẾ VÀ TỐI ƯU CHO VGG-TINY

Thay vì cố gắng tổng quát hóa hỗ trợ tất cả các loại mạng CNN, đề tài chọn **thu hẹp tập op** mà bộ tăng tốc hỗ trợ trong phiên bản v2.0 alpha để đảm bảo tiến độ và tính đúng đắn:

- Hỗ trợ Conv 3×3 với padding VALID, stride 1, hàm kích hoạt ReLU (fused).
- Hỗ trợ tới `MAX_CIN` = 64 và `MAX_COUT` = 64 kênh trên mỗi layer (tham số hóa trong `conv_pkg.sv`).
- Hỗ trợ runtime-configurable cho geometry ($W \times H$), số kênh, hệ số lượng tử hóa qua thanh ghi điều khiển AXI4-Lite.

Lựa chọn này dẫn đến mô hình triển khai chính là `vgg-tiny` – một mạng *fully-convolutional* đơn giản chỉ dùng các op nói trên (mục 4.7.1). Các kiến trúc phức tạp hơn như Pointwise 1×1, Depthwise, Stride 2, Padding SAME, hay Residual Add được dành cho các pha mở rộng tương lai trong roadmap (mục 4.7.3).

4.7. MÔ HÌNH AI MỤC TIÊU: VGG-TINY

Việc lựa chọn mô hình AI chạy trên hệ thống phải cân bằng giữa hai yếu tố: (i) tập op mà bộ tăng tốc Conv-CNN v2.0 hỗ trợ (xem mục 4.6.3) và (ii) độ chính xác đủ tốt cho bài toán phân loại nhị phân. Đề tài đề xuất một kiến trúc tinh gọn riêng tên `vgg-tiny` – lấy cảm hứng từ phong cách VGG (chỉ dùng Conv 3×3 stacking) nhưng được rút gọn để chạy được hoàn toàn trên phần cứng đã thiết kế.

4.7.1. ĐẶC TẢ KIẾN TRÚC VGG-TINY

`vgg-tiny` là một mạng *fully-convolutional* 5 layer, đầu vào ảnh RGB 128×128, output là tensor 2 kênh được ARM thực hiện Global Average Pooling + softmax + argmax thành nhãn cuối cùng. Bảng 4.4 mô tả cấu trúc chi tiết.

Bảng 4.4: Cấu trúc chi tiết các lớp mạng vgg-tiny (đầu vào $128 \times 128 \times 3$, lượng tử hóa INT8)

Layer	Loại op	IFM	OFM	Kernel	Activation
L0	Conv2D + ReLU	$128 \times 128 \times 3$	$126 \times 126 \times 8$	3×3	ReLU
L1	Conv2D + ReLU	$126 \times 126 \times 8$	$124 \times 124 \times 16$	3×3	ReLU
L2	Conv2D + ReLU	$124 \times 124 \times 16$	$122 \times 122 \times 32$	3×3	ReLU
L3	Conv2D + ReLU	$122 \times 122 \times 32$	$120 \times 120 \times 64$	3×3	ReLU
L4	Conv2D (logits)	$120 \times 120 \times 64$	$118 \times 118 \times N$	3×3	None
Hậu xử lý trên ARM: GAP → softmax → argmax → nhãn (N = số lớp = 2)					

Các đặc điểm chính:

- **Tất cả op đều thuộc tập hỗ trợ của bộ tăng tốc:** Conv 3×3, padding VALID, stride 1, ReLU fused. Layer cuối cùng (`L4`) tắt activation để xuất logits cho softmax trên ARM.

- **Không có Dense head:** Layer cuối là Conv 3×3 với `cout` = N (số lớp), thay vì Conv → Flatten → Dense kiểu VGG truyền thống. Cách này loại bỏ nhu cầu phần cứng hỗ trợ Fully Connected layer.

- **Không có MaxPool** trong v2.0 alpha: kích thước feature map giảm dần qua valid-padding (-2 pixel mỗi chiều). Kết quả là feature map cuối tương đối lớn (118×118) và latency cao hơn so với phiên bản có pool. Việc bổ sung MaxPool 2×2 hoặc Conv stride-2 được dành cho V2.1.

- **Số tham số nhỏ:** Tổng cộng ≈ 25,700 tham số INT8 (xem chi tiết tại bảng **??** trong chương 6), dung lượng weights blob khoảng 25 KB – nằm gọn trong bộ nhớ DDR và dễ quản lý.

4.7.2. SO SÁNH VGG-TINY VỚI CÁC MÔ HÌNH THÔNG DỤNG

Để minh chứng cho lựa chọn thiết kế, bảng 4.5 so sánh `vgg-tiny` với các kiến trúc CNN phổ biến trên cùng tiêu chí:

Bảng 4.5: So sánh vgg-tiny với các mạng CNN phổ biến

Mô hình	Input	Số layer	Params	INT8 size	MACs	FPGA-deployable
`vgg-tiny`	128 × 128	5	25.7K	25.1 KB	371 M	✓
VGG11 / VGG16	224 × 224	13	14.7M	14.4 MB	15,347 M	× (FC head + pool stride-2)
ResNet-18 (V2)	224 × 224	53	23.6M	23.0 MB	3,480 M	× (1×1 + BN + residual)
EfficientNet-B0	224 × 224	81	4.05M	3.96 MB	385 M	× (depthwise + SE + swish)
MobileNetV1/V2	224 × 224	28	2.3–4.2M	2.2–4.0 MB	300–569 M	× (depthwise)

So với các mô hình thương mại, `vgg-tiny` nhỏ gọn hơn từ 100× đến 900× về số tham số, đồng thời tránh được hoàn toàn các op khó (depthwise, BN, residual, SE). Đánh đổi là không gian biểu diễn (representation capacity) hạn chế, phù hợp với bài toán phân loại nhị phân nhưng chưa đủ cho bài toán phân loại nhiều lớp phức tạp như ImageNet 1000 classes.

4.7.3. LỘ TRÌNH MỞ RỘNG (TIER ROADMAP)

Đề tài định hướng triển khai mở rộng theo ba tier, với `vgg-tiny` là Tier 1 (đã xong), Tier 2–3 là các pha mở rộng:

- **Tier 1 – vgg-tiny (đã triển khai):** Chỉ Conv 3×3 stride-1 valid + ReLU. Bộ tăng tốc v2.0 alpha.
- **Tier 2 – VGG11 / ResNet-18 (mở rộng):** Bổ sung MaxPool 2×2 (hoặc stride-2 conv), padding SAME, 1×1 conv, residual addition. Yêu cầu mở rộng `conv_core.sv` và `controller_fsm.sv` nhưng không thay đổi giao thức AXI hay layer table contract.
- **Tier 3 – ResNet-50 / YOLOv3-Tiny (stretch):** Thêm BN folding offline, leaky-ReLU, upsample 2× cho detection head.

Mọi mở rộng đều **không phá vỡ ABI v2.0**: chỉ cần thêm enum cho `activation_t/padding_t/kernel_size` trong `layer_desc.h` và thêm 1 mode-bit ở conv_core MUX. Pipeline huấn luyện và phần mềm host không cần thay đổi.

4.7.4. LƯỢNG TỬ HÓA (QUANTIZATION)

Vi xử lý RISC-V và Khối tăng tốc phần cứng trên FPGA được thiết kế để xử lý số nguyên (Integer Arithmetic) nhằm tối đa hóa tốc độ và tiết kiệm bộ nhớ. Do đó, mô hình gốc (Float32) cần trải qua quy trình *Post-training Quantization*:

- **Weights (Trọng số):** Chuyển đổi từ `float32` (4 bytes) sang `int8` (1 byte).
- **Activations (Giá trị kích hoạt):** Đầu ra của mỗi lớp cũng được ép về `int8`.
- **Bias (Hệ số chệch):** Lưu trữ dưới dạng `int32` để tránh tràn số (overflow) khi cộng dồn, sau đó được dịch bit (bit-shift) để quay về `int8`.

Công thức lượng tử hóa tổng quát được áp dụng:

$$R = S(Q - Z) \tag{4.2}$$

Trong đó R là giá trị thực (Real value), Q là giá trị lượng tử hóa (Quantized value), S là tỷ lệ (Scale) và Z là điểm không (Zero-point).

4.7.5. TỔ CHỨC DỮ LIỆU TRONG BỘ NHỚ

Để tối ưu hóa cho việc truy cập Burst qua AXI4 Bus (128-bit width), các ma trận trọng số (Weights) không được lưu trữ theo thứ tự tuyến tính thông thường mà được sắp xếp lại (Memory Packing):

- **Kernel Layout:** Các bộ lọc được sắp xếp theo cấu trúc `OHWI` (Output-Height-Width-Input) để đảm bảo khi uDMA đọc 16 bytes (128-bit) sẽ thu được đủ dữ liệu cho các bộ nhân hoạt động song song.
- **C-Header Generation:** File mô hình `.tflite` cuối cùng được chuyển đổi (dump) thành các mảng hằng số trong ngôn ngữ C (`weights.h`) để biên dịch cùng với Firmware của RISC-V.

4.8. LUỒNG THỰC THI VÀ GIAO THỨC BẮT TAY

Trong kiến trúc dị thể, việc đồng bộ hóa giữa vi xử lý ARM (chạy Linux + PYNQ runtime) và lõi RISC-V (chạy bare-metal) là yếu tố quyết định tính đúng đắn của hệ thống. Đề tài sử dụng cơ chế bắt tay dựa trên các thanh ghi trạng thái trong D-BRAM Port B (đã trình bày tại mục 4.4.2). Phần này mô tả luồng thực thi end-to-end cho một lần inference.

Quy trình inference một ảnh Quá trình thực thi diễn ra qua 4 pha tuần tự, được minh họa trong sơ đồ tuần tự Hình 4.6:

1. **Pha 1 – ARM chuẩn bị dữ liệu:**
   - ARM đọc ảnh `.jpg/.png` từ thẻ nhớ SD bằng `cv2.imread`.
   - Tiền xử lý: chuyển BGR→RGB, resize về 128×128, chuẩn hóa $[0, 1]$.
   - Lượng tử hóa INT8 theo công thức $q = \text{round}(x/S_{\text{in}}) + Z_{\text{in}}$ với $S_{\text{in}}, Z_{\text{in}}$ đọc từ `meta.json`.
   - Cấp phát ba CMA buffer qua `pynq.allocate`: `wbuf` (weights), `buf_a` (IFM ping-pong), `buf_b` (scratch ping-pong); ghi pixel ảnh đã lượng tử vào `buf_a` và gọi `flush()`.
   - Ghi 3 thanh ghi địa chỉ vật lý `IFM_PHYS_ADDR`, `OFM_PHYS_ADDR`, `WEIGHT_BASE` và `DATASET_ID` vào D-BRAM Port B (mục 4.4.2).
2. **Pha 2 – ARM kích hoạt RISC-V:**
   - Ghi `0x01` (START) vào `CMD_FROM_ARM @ 0xB004_0000`.
   - ARM chuyển sang chế độ chờ (polling `STATUS_TO_ARM` hoặc đợi IRQ).
3. **Pha 3 – RISC-V điều phối tính toán:**
   - RISC-V phát hiện `CMD_FROM_ARM == 0x01`, ghi `STATUS_TO_ARM = 0x01` (BUSY).
   - Duyệt mảng `LAYERS[]` (compile-in trong I-BRAM): với mỗi layer i:
     - Tính địa chỉ vật lý weight = `WEIGHT_BASE + LAYERS[i].weight_offset`; ping-pong: `in = (i&1)?buf_b:buf_a`, `out = ngược lại`.
     - Cấu hình `conv_cnn_0/S00_AXI`: ghi geometry, channels, M_{q31}, shift, zero-point, có ReLU; chuyển sang chế độ load weight.
     - Lập trình `axi_dma_0`: MM2S đọc weights từ DDR → `conv_cnn_0/S00_AXI` chờ done.
     - Chuyển sang chế độ infer; lập trình DMA tiếp: MM2S đọc IFM, S2MM ghi OFM về DDR; chờ done.
   - Sau layer cuối, ghi `STATUS_TO_ARM = 0x02` (DONE).
4. **Pha 4 – ARM hậu xử lý:**

- ARM phát hiện `STATUS_TO_ARM == 0x02`, gọi `.invalidate()` trên buffer chứa OFM cuối (xác định bởi `final_ofm_buf_idx` trong `meta.json`).
- Thực hiện Global Average Pool trên trục (H, W) của tensor INT8 OFM, sau đó dequantize về float, áp dụng softmax, lấy argmax để có nhãn cuối cùng.
- Hiển thị kết quả lên notebook (matplotlib overlay), in ra console hoặc lưu file log.
- Ghi `CMD_FROM_ARM = 0x00` (IDLE) để sẵn sàng cho ảnh tiếp theo.

chapter04/img/flow.png

Hình 4.6: Sơ đồ quy trình

5

HIỆN THỰC VÀ KIỂM THỬ HỆ THỐNG

Chương này trình bày chi tiết quá trình hiện thực hóa kiến trúc phần cứng đã đề xuất ở các chương trước thành hệ thống vật lý trên nền tảng FPGA. Nội dung bao gồm việc tinh chỉnh lõi vi xử lý RISC-V, thiết kế cầu nối giao tiếp, tổng hợp bộ gia tốc phần cứng cho mạng nơ-ron, lập trình phần mềm nhúng điều khiển và quy trình huấn luyện, lượng tử hóa mô hình trí tuệ nhân tạo.

5.1. TỔNG QUAN HƯỚNG HIỆN THỰC

Hệ thống gia tốc AI tại biên (Edge-AI) trong đồ án này được thiết kế theo phương pháp đồng thiết kế phần cứng/phần mềm (Hardware-Software Co-design) mang tính dị cấu trúc (Heterogeneous). Để đảm bảo tính linh hoạt (linh hoạt thay đổi mô hình AI mà không cần tổng hợp lại phần cứng) và hiệu năng cao, hệ thống được hiện thực hóa theo mô hình phân tầng kiến trúc (Layered Architecture) gồm 4 tầng (Layer) phối hợp chặt chẽ với nhau.

chapter05/img/flowwork.png

Hình 5.1: Sơ đồ tổng quan

Quy ước giao tiếp giữa các tầng

Bảng 5.1: Các artifact trao đổi giữa các tầng trong pipeline.

Artifact	Sinh bởi	Tiêu thụ bởi
`firmware.bin`	RISC-V toolchain	ARM (load vào I-BRAM)
`weights.bin`	training/train.py	ARM (load vào DDR)
`meta.json`	training/train.py	ARM (input quantization)
`layer_table.h`	training/train.py	RISC-V firmware (compile-in)
`bitstream.bit`	Vivado synth+impl	PYNQ Overlay

5.2. HUẤN LUYỆN VÀ LƯỢNG TỬ HÓA MÔ HÌNH AI

Mô hình trí tuệ nhân tạo được sử dụng trong hệ thống là mạng nơ-ron tích chập (CNN). Quy trình xử lý mô hình được thực hiện hoàn toàn ngoại tuyến (Offline) trên máy tính cá nhân qua các bước:

- **Huấn luyện mô hình (Training):** Sử dụng thư viện học máy phổ biến (TensorFlow/Keras) để xây dựng và huấn luyện mô hình phân loại đối tượng trên các tập dữ liệu tiêu chuẩn. Mô hình sử dụng kiểu dữ liệu số thực độ chính xác 32-bit (FP32) truyền thống để đạt được độ chính xác cao nhất trong quá trình hội tụ.

- **Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ):** Do bộ gia tốc phần cứng được thiết kế cho kiểu dữ liệu số nguyên INT8 nhằm tiết kiệm tài nguyên, toàn bộ trọng số (Weights) và các tham số kích hoạt (Activations) của mô hình FP32 được ánh xạ và ép kiểu về INT8 thông qua bộ cung cấp dữ liệu hiệu chuẩn (Calibration Dataset). Quá trình này giúp giảm 4 lần dung lượng lưu trữ mô hình và tăng tốc độ suy luận phần cứng mà vẫn đảm bảo độ hao hụt độ chính xác ở mức cho phép.

- **Xuất dữ liệu:** Các trọng số và siêu tham số sau khi lượng tử hóa được xuất ra dưới dạng các mảng dữ liệu ngôn ngữ C (Header files), sẵn sàng để biên dịch cùng mã nguồn Firmware và nạp trực tiếp vào bộ nhớ của hệ thống SoC.

5.3. HIỆN THỰC KHỐI GIA TỐC PHẦN CỨNG CNN

Khắc phục lỗi suy luận BRAM"từ 300,000 FDRE xuống còn 9 BRAM nhờ làm phẳng mảng và bỏ lệnh Reset.

5.3.1. TỔNG QUAN KIẾN TRÚC CONV_CNN V2.0

Hình 5.2: Sơ đồ datapath conv_cnn IP: từ AXIS in qua line buffer, window, MAC array, requantize đến AXIS out.

5.3.2. THAM SỐ HÓA VÀ PACKAGE `conv_pkg.sv`

Toàn bộ tham số bound (`MAX_WIDTH`, `MAX_CIN`, `MAX_COUT`) và độ rộng datatype được đặt trong `conv_pkg.sv`, cho phép scale tài nguyên mà không sửa

từng module.

5.3.3. TÍCH HỢP TỔNG THỂ QUA AXI

Bảng 5.2: Bản đồ thanh ghi điều khiển S00_AXI (32 bytes, 8 thanh ghi).

Offset	Tên	R/W	Layout
0x00	GEOMETRY	RW	width[15:0], height[31:16]
0x04	CTRL	RW	start, pool_en, mode_load, has_relu
0x08	STATUS	RO	done
0x0C	CHANNELS	RW	num_cin[15:0], num_cout[31:16]
0x10	M_Q31	RW	Q31 multiplier
0x14	SHIFT_ZP	RW	shift[5:0], output_zp[15:8]

5.3.4. ĐÁNH GIÁ KHỐI TĂNG TỐC CONV-CNN

Hình 5.3: Báo cáo chạy khối CNN

5.4. HIỆN THỰC RISC-V SOFT-CORE

5.4.1. TINH CHỈNH CV32E40P CORE

Lõi vi xử lý CV32E40P mã nguồn mở nguyên bản được thiết kế với đầy đủ các tập lệnh mở rộng, bao gồm cả khối xử lý số thực dấu phẩy động (FPU - Floating Point Unit). Tuy nhiên, để tối ưu hóa tài nguyên phần cứng (LUT, FF, DSP) trên chip FPGA cho bộ tăng tốc trí tuệ nhân tạo, lõi CPU đã được tinh chỉnh lại ở mức mã nguồn RTL.

Cụ thể, toàn bộ khối xử lý số thực và các lệnh liên quan (tập lệnh mở rộng F) đã được loại bỏ. Sự tinh giản này là hoàn toàn hợp lý do toàn bộ mô hình học sâu trong hệ thống đã được lượng tử hóa xuống kiểu dữ

Hình 5.4: Kết quả chạy simulate khối CNN

liệu số nguyên 8-bit (INT8). Vi xử lý lúc này chỉ hoạt động với tập lệnh cơ bản RV32IMC, chịu trách nhiệm điều phối luồng điều khiển, quản lý ngắt và cấu hình thanh ghi thay vì tham gia tính toán trực tiếp. Bên cạnh đó, bộ giải mã lệnh (Instruction Decoder) cũng được tùy biến để tối ưu hóa đường truyền dữ liệu, loại bỏ các khối logic không cần thiết, giúp cải thiện tần số hoạt động tối đa (Fmax) của lõi.

5.4.2. HIỆN THỰC CẦU NỐI GIAO TIẾP OBI SANG AXI4 (OBI TO AXI)

Giao thức giao tiếp bộ nhớ mặc định của lõi CV32E40P là OBI (Open Bus Interface). Tuy nhiên, nền tảng thiết kế SoC của Xilinx Vivado sử dụng

chuẩn giao tiếp AMBA AXI4 làm nòng cốt. Để tích hợp thành công lõi RISC-V vào bản vẽ hệ thống cùng với khối xử lý trung tâm (PS) và các bộ nhớ đệm, một cầu nối giao tiếp (Bridge) chuyển đổi từ tín hiệu OBI sang AXI4 đã được thiết kế và hiện thực.

Cầu nối này (obi_to_axi) bao gồm hai luồng độc lập: một luồng dành cho việc nạp lệnh (Instruction Fetch) và một luồng dành cho truy xuất dữ liệu (Data Access). Khối logic bên trong thực hiện nhiệm vụ biên dịch các tín hiệu điều khiển (như tín hiệu yêu cầu *req*, tín hiệu cấp phép *gnt*) của chuẩn OBI thành các kênh địa chỉ, dữ liệu và phản hồi tương ứng của chuẩn AXI4-Lite/AXI4 Full. Quá trình kiểm thử mô phỏng (Simulation) đảm bảo cầu nối không gây ra trạng thái tắc nghẽn (Deadlock) và xử lý chính xác các tín hiệu báo lỗi đường truyền.

5.5. PHÁT TRIỂN FIRMWARE

5.5.1. PIPELINE HUẤN LUYỆN TRÊN HOST PC

Listing 5.1: Kiến trúc Tiny-VGG đề xuất

```
def build_vgg_tiny(num_classes, input_shape=(128,128,3)):
    inputs = tf.keras.Input(shape=input_shape)
    x = layers.Conv2D(8,  3, activation='relu', padding='valid
    x = layers.Conv2D(16, 3, activation='relu', padding='valid
    x = layers.Conv2D(32, 3, activation='relu', padding='valid
    x = layers.Conv2D(64, 3, activation='relu', padding='valid
    x = layers.Conv2D(num_classes, 3, activation=None, padding
    x = layers.GlobalAveragePooling2D()(x)
    return models.Model(inputs, layers.Softmax()(x), name='vgg
```

5.5.2. TRÌNH BIÊN DỊCH TỰ SINH `layer_table.h` VÀ `weights.bin`

$$M_{q31}, n = \texttt{quantize_multiplier}\left(\frac{s_{\text{in}} \cdot s_{\text{w}}}{s_{\text{out}}}\right) \tag{5.1}$$

5.5.3. FIRMWARE RISC-V

linked.ld startup.s

5.5.4. HOST CONTROLLER

Jupiter Notebook server

5.6. TÍCH HỢP BLOCK DESIGN TRONG VIVADO

Hình 5.5: Sơ khối minh họa hệ thống

Hình 5.6: Thiết kế khối trong vivado

5.6.1. BẢN ĐỒ BỘ NHỚ VÀ ADDRESS EDITOR

5.6.2. ĐỒNG BỘ ARM ↔ RISC-V QUA D-BRAM DUAL-PORT

5.7. QUY TRÌNH KIỂM THỬ

5.7.1. KIỂM THỬ ĐƠN VỊ (UNIT TESTBENCH)

5.7.2. KIỂM THỬ TÍCH HỢP IP

Testbench `tb_conv_core.sv` mô phỏng toàn bộ luồng load + infer với input 4×4×1, weights toàn 1, M=0.5, kỳ vọng OFM = $\{23, 27, 41, 45\}$ (với round-half-up).

$$y_q = \mathrm{sat}_{\mathrm{INT8}}\left(\left\lfloor \frac{(\sum_{i,j,c} x \cdot w + b) \cdot M_{q31} + 2^{30+n}}{2^{31+n}} \right\rfloor + z_{\mathrm{out}}\right) \tag{5.2}$$

5.7.3. ĐỐI CHIẾU VỚI GOLDEN VECTOR TỪ TFLITE INTERPRETER

5.7.4. KIỂM THỬ TRÊN BOARD KV260

6

KẾT QUẢ

Trong chương này, nhóm thực hiện trình bày các kết quả đạt được của đồ án, bao gồm hai phần chính: Đánh giá quá trình tổng hợp phần cứng (Hardware Synthesis) trên phần mềm Xilinx Vivado và đánh giá hiệu năng huấn luyện, lượng tử hóa các mô hình Mạng nơ-ron tích chập (CNN) trước khi triển khai thực tế xuống bo mạch Kria KV260.

6.1. KẾT QUẢ HUẤN LUYỆN MODEL

6.1.1. CẤU HÌNH THÍ NGHIỆM

- Hardware huấn luyện: GPU NVIDIA RTX/A100/...
- Framework: TensorFlow 2.x + Keras
- Optimizer: Adam, learning rate 10^{-3}
- Batch size, epochs, augmentation...

6.1.2. TẬP CATS VS DOGS

Bảng 6.1: Accuracy Tiny-VGG trên tập Cats vs Dogs.

Model	Train Acc	Val Acc	Test Acc
FP32 (reference)	...	...	...
INT8 (TFLite PTQ)	...	...	...
INT8 + FPGA	...	...	...

Hình 6.1: Đường cong loss và accuracy theo epoch.

6.1.3. TẬP INRIA PERSON

6.1.4. ĐÁNH GIÁ ẢNH HƯỞNG CỦA LƯỢNG TỬ HÓA INT8

6.2. TỔNG HỢP PHẦN CỨNG

Quá trình tổng hợp (Synthesis) và thực thi định tuyến (Implementation) hệ thống SoC bao gồm lõi vi xử lý ARM, lõi softcore RISC-V và bộ gia tốc phần cứng CNN được thực hiện trên công cụ Vivado. Các thông số về tài nguyên, công suất và thời gian (timing) được trích xuất để đảm bảo hệ thống đáp ứng được yêu cầu thiết kế.

Hình 6.2: Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic)

6.2.1. TÀI NGUYÊN FPGA

Sau quá trình Place & Route, hệ thống được kiểm tra luật thiết kế để đảm bảo không xảy ra xung đột tài nguyên vật lý, đặc biệt là tại khu vực cấp

Bảng 6.2: Tài nguyên KV260 sử dụng (sau implementation).

Resource	**Used**	**Available**	**Util.**
LUT	...	117120	...%
FF	...	234240	...%
BRAM	...	144	...%
DSP	...	1248	...%
URAM	...	64	...%

Hình 6.3: Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic)

phát Dual-Port BRAM cho kiến trúc bộ nhớ Harvard.

Hình 6.4: Sơ đồ khối tổng thể của hệ thống sau khi tổng hợp (Schematic)

6.2.2. BÁO CÁO TIMING

Hình 6.5: Báo cáo kiểm tra luật thiết kế (DRC Report)

6.2.3. POWER ESTIMATION

Hình 6.6: Báo cáo thời gian (Timing Summary) đảm bảo không có vi phạm Slack

6.3. KẾT QUẢ CHẠY TRÊN MẠCH THỰC TẾ

Các mô hình sau khi đạt được độ chính xác mục tiêu trên máy tính (PC) được biên dịch thành các tệp trọng số nhị phân ('.bin') và nạp xuống FPGA. Lõi vi xử lý ARM thông qua thư viện PYNQ đóng vai trò điều phối luồng dữ liệu (DMA) và kích hoạt vi điều khiển RISC-V để điều khiển bộ gia tốc phần cứng tiến hành suy diễn.

6.3.1. SO SÁNH VỚI RELATED WORK

Hình 6.7: Báo cáo ước tính công suất tiêu thụ (Power Report) của toàn hệ thống

7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. KẾT LUẬN

7.1.1. ĐẠT ĐƯỢC

Đồ án đã nghiên cứu và triển khai thành công một hệ thống vi mạch nhúng SoC (System-on-Chip) hoàn chỉnh dựa trên phương pháp đồng thiết kế Phần cứng - Phần mềm (Hardware-Software Co-design). Hệ thống được hiện thực hóa trên nền tảng FPGA Xilinx Kria KV260 (Zynq UltraScale+), hướng tới mục tiêu gia tốc các thuật toán Trí tuệ nhân tạo tại biên (Edge AI).

Thông qua quá trình thực hiện, nhóm sinh viên đã đạt được những kết quả cốt lõi sau:

- **Xây dựng thành công kiến trúc xử lý Đa nhân (Dual-core):** Kết hợp linh hoạt giữa sức mạnh quản lý cấp cao của lõi ARM (chạy hệ điều hành Linux/thư viện PYNQ) và khả năng xử lý thời gian thực, tiêu thụ năng lượng thấp của vi điều khiển mã nguồn mở RISC-V (chạy Bare-metal).

- **Tối ưu hóa kiến trúc bộ nhớ Harvard trên FPGA:** Quy hoạch thành công hệ thống bộ nhớ Dual-Port BRAM với 64KB I-BRAM (Bộ nhớ lệnh) và 256KB D-BRAM (Bộ nhớ dữ liệu). Việc thiết lập 4 khối AXI BRAM Controller độc lập đã loại bỏ triệt để hiện tượng nghẽn cổ chai (bottleneck) khi ARM và RISC-V cùng truy xuất bộ nhớ.

- **Khắc phục độ trễ dữ liệu bằng AXI HPC:** Triển khai thành công bộ điều khiển truy cập bộ nhớ trực tiếp (AXI DMA) kết nối qua cổng S_AXI_HPC. Điều này giúp tối đa hóa băng thông truyền tải ảnh/trọng số từ RAM DDR vào BRAM và tự động đồng bộ hóa bộ đệm (Cache Coherency) mà không cần can thiệp phức tạp bằng phần mềm.

- **Lượng tử hóa và chuẩn hóa mô hình CNN:** Thực hiện thành công kỹ thuật Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ), ép kiểu các mô hình Mạng nơ-ron tích chập (YOLO-Fastest, ResNet, VGG) từ dấu phẩy động 32-bit (FP32) xuống số nguyên 8-bit (INT8). Giải pháp này giúp giảm đáng kể dung lượng lưu trữ và tài nguyên phần cứng mà vẫn duy trì được độ chính xác có thể chấp nhận được trên các tập dữ liệu thực tế như INRIA Person và Dogs vs. Cats.

Nhìn chung, đồ án đã chứng minh tính khả thi và hiệu quả của việc tự thiết kế một bộ gia tốc AI chuyên dụng (Custom AI Accelerator) giao tiếp với softcore RISC-V trên nền tảng FPGA, thay vì phụ thuộc hoàn toàn vào các vi xử lý thương mại có sẵn.

7.1.2. HẠN CHẾ CỦA HỆ THỐNG

- Chỉ hỗ trợ 3×3 conv stride 1 valid padding (chưa có 1×1, depthwise, padding)

- Per-tensor quantization (chưa per-axis)

- Throughput còn hạn chế do all-serial cnt_cout

7.2. HƯỚNG PHÁT TRIỂN

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng được các mục tiêu ban đầu, đồ án vẫn còn nhiều không gian để tối ưu hóa và mở rộng trong tương lai. Một số hướng phát triển cụ thể bao gồm:

- **Tối ưu hóa luồng dữ liệu bằng Ping-Pong Buffering:** Mở rộng logic điều khiển DMA và RISC-V để áp dụng kỹ thuật chia gạch (Tiling) và đệm bóng bàn (Ping-Pong Buffering) trong D-BRAM. Điều này sẽ giúp che giấu hoàn toàn độ trễ truyền dữ liệu (Latency Hiding), cho phép khối CNN tính toán liên tục không có thời gian chết.

- **Mở rộng tập lệnh RISC-V (Custom Instructions):** Tận dụng đặc tính mã nguồn mở của RISC-V để can thiệp vào bộ giải mã lệnh (Instruction Decoder), tạo ra các tập lệnh tùy chỉnh (Custom Instructions) dành riêng cho việc cấu hình và kích hoạt khối gia tốc CNN thay vì dùng giao tiếp Memory-Mapped I/O thông thường.

- **Tích hợp Hệ điều hành thời gian thực (RTOS):** Nâng cấp phần mềm chạy trên lõi RISC-V từ dạng Bare-metal (vòng lặp vô tận) lên một hệ điều hành thời gian thực nhỏ gọn như FreeRTOS. Điều này sẽ giúp hệ thống có khả năng quản lý nhiều tác vụ đồng thời (nhận dữ liệu cảm biến, điều khiển động cơ, chạy AI) với độ trễ phản hồi xác định.

- **Mở rộng hỗ trợ mô hình học sâu:** Nâng cấp kiến trúc phần cứng của khối tính toán tích chập để hỗ trợ thêm các lớp mạng phức tạp hơn như Depthwise Separable Convolution (trong MobileNet) hoặc cơ chế Attention (trong Vision Transformers), từ đó nâng cao độ chính xác cho các bài toán thị giác máy tính phức tạp.

- **Xây dựng giao diện người dùng (Web UI):** Tận dụng hệ điều hành Ubuntu chạy trên lõi ARM để phát triển một máy chủ Web cục bộ. Máy chủ này sẽ tiếp nhận luồng Video trực tiếp từ USB Camera, gọi module gia tốc phần cứng dưới PL để suy diễn, và hiển thị kết quả Bounding Box trực quan lên trình duyệt web theo thời gian thực.

TÀI LIỆU THAM KHẢO

[1] D. Patterson **and** J. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface* (The Morgan Kaufmann Series in Computer Architecture and Design). Morgan Kaufmann, 2020, ISBN: 9780128245583. **url**: https://books.google.com.vn/books?id=e8DvDwAAQBAJ.

[2] L. H. Crockett, D. Northcote, C. Ramsay, F. D. Robinson **and** R. W. Stewart, *Exploring Zynq MPSoC: With PYNQ and Machine Learning Applications*. Glasgow, UK: Strathclyde Academic Media, 2019, Available online at https://www.zynq-mpsoc-book.com/, ISBN: 978-0-9929787-3-0. **url**: https://www.zynq-mpsoc-book.com/.

[3] *Introduction to AMBA AXI4*, Accessed: May 1, 2026, Arm Limited, 2020. **url**: https://developer.arm.com.

[4] A. Munir, J. Kong **and** M. A. Qureshi, *Accelerators for Convolutional Neural Networks*, 1 **edition**. Hoboken, NJ: Wiley-IEEE Press, 2024, **page** 304, ISBN: 978-1-394-17188-0. DOI: 10.1002/9781394171910. **url**: https://ieeexplore.ieee.org/book/10296338.

[5] P. D. Schiavone, D. Rossi, A. Pullini, A. Di Mauro, F. Conti **and** L. Benini, "Quentin: an Ultra-Low-Power PULPissimo SoC in 22nm FDX," **in***2018 IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)* 2018, **pages** 1–3. DOI: 10.1109/S3S.2018.8640145.

[6] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv **and** Y. Bengio, "Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1," **in***Advances in Neural Information Processing Systems (NIPS)* **volume** 29, 2016.

[7] B. Jacob, S. Kligys, B. Chen **andothers**, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," **in***Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 2018, **pages** 2704–2713.

[8] A. G. Howard, M. Zhu, B. Chen **andothers**, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017.

[9] A. Parameshwara **and** S. H. Mokashi, *FPGA-Accelerated RISC-V ISA Extensions for Efficient Neural Network Inference on Edge Devices*, arXiv preprint arXiv:2511.06955, 2025. arXiv: `2511.06955 [cs.AR]`.

[10] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao **and** J. Cong, "Caffeine: A Hardware-Software Co-Design Library for Deep Learning on FPGA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, **jourvol** 37, **number** 11, **pages** 2266–2279, 2018. DOI: `10.1109/TCAD.2017.2785257`.

[11] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott **andothers**, "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," **in***Exhamining the Interface Between High-Performance Computing and Big Data* FPGA Symposium, 2017.